Data Science Midterm

Introduction

Within the dynamic field of data science, the use of linear regression is a powerful technique for identifying patterns within particular domains. The automotive industry has been selected as the domain for this coursework. This sector is characterized by complex dynamics and a wide range of variables that affect one of its primary elements, automobile price. This investigation aims to uncover deep insights by using machine learning approaches, primarily concentrating on linear regression. The main goal is to develop a model that will be useful to industry stakeholders and customers alike in addition to understanding the fundamental dynamics of auto pricing. Given its intrinsic complexity and the multitude of elements impacting pricing strategies, the automotive sector shows to be a perfect subject for the use of linear regression. Preprocessing, EDA, Visualizations, and finally the Linear Regression Model are the phases involved in applying linear regression.

Domain-Specific Area: Automotive Industry

Because of its diversity, the automobile industry provides a wealth of research opportunities. Numerous elements, including the manufacturer, model, year, engine specs, and other characteristics, affect car prices. To better understand price dynamics, it is possible to identify underlying patterns and correlations among these variables by utilizing a linear regression model in this domain.

Objectives of the Project

1.Unraveling Pricing Patterns

Deciphering the complex pricing patterns in the automotive sector is one of the main goals. Using a linear regression model, our goal is to pinpoint the essential characteristics that are crucial in deciding an automobile's final cost. Through investigation, it may be possible to find connections and patterns that have been overlooked by traditional analysis.

2.Informed Decision-Making for Consumers

With the help of knowledge that goes beyond simple market trends, the project aims to empower consumers. Potential automobile purchasers may find it useful to have a solid linear regression model as a tool for making data-driven judgments. This is in line with the overarching objective of improving consumer choice in a convoluted marketplace.

3.Industry Stakeholder Guidance

The project provides manufacturers and dealers, among other industry stakeholders, with a window into the variables affecting pricing tactics. Comprehending the ways in which different characteristics influence an automobile's perceived value facilitates more astute choices about marketing, production, and inventory control.

Dataset description

The dataset used for this assignment comes from the well-known website Kaggle, which provides a wealth of datasets for a variety of fields. The dataset provides a solid basis for our investigation into linear regression modeling by encapsulating a diverse range of information pertaining to different automotive features. It combines information from automakers, dealers, and market research to create a unified database covering several aspects of the automotive sector. Using Kaggle guarantees access to premium datasets selected by a varied group of professionals and data enthusiasts. Make, Model, Year, Engine Fuel Type, Engine HP, Engine Cylinders, Transmission Type, Driven Wheels, Number of Doors, Market Category, Vehicle Size, Vehicle Style, Highway MPG, City MPG, and Popularity are some of the columns that made it a great fit for Linear Regression. The dataset's 10,000 entries allowed for a thorough investigation without sacrificing statistical significance. The dataset's combination of numerical and categorical variables reflects the variety of vehicle features in terms of data kinds.

Dataset Preparation

First, we will import the required libraries and will then preprocess the data as required. I have loaded the csv file.

In [299…

```
#code that i have written
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

In [300…

```
data_car = pd.read_csv('data.csv')
data_car.head()
```

Out[300]:

	Make	Model	Year	Engine Fuel Type	Engine HP	Engine Cylinders	Transmission Type	Driven_Wheels	Numbe o Door
0	BMW	1 Series M	2011	premium unleaded (required)	335.0	6.0	MANUAL	rear wheel drive	2.
1	BMW	1 Series	2011	premium unleaded (required)	300.0	6.0	MANUAL	rear wheel drive	2.
2	BMW	1 Series	2011	premium unleaded (required)	300.0	6.0	MANUAL	rear wheel drive	2.
3	BMW	1 Series	2011	premium unleaded (required)	230.0	6.0	MANUAL	rear wheel drive	2.
4	BMW	1 Series	2011	premium unleaded (required)	230.0	6.0	MANUAL	rear wheel drive	2.

In [301…

```
data_car.sample(10)
```

Out[301]:

	Make	Model	Year	Engine Fuel Type	Engine HP	Engine Cylinders	Transmission Type	Driv
8468	Buick	Rendezvous	2006	regular unleaded	195.0	6.0	AUTOMATIC	all
1895	Volkswagen	Beetle	2015	premium unleaded (recommended)	210.0	4.0	MANUAL	
7755	Audi	Q3	2016	premium unleaded (recommended)	200.0	4.0	AUTOMATIC	all
1498	Cadillac	ATS Coupe	2015	flex-fuel (unleaded/E85)	321.0	6.0	AUTOMATIC	
9731	Hyundai	Sonata	2016	regular unleaded	245.0	4.0	AUTOMATIC	
3071	Honda	CR-Z	2014	regular unleaded	130.0	4.0	MANUAL	
8302	Land Rover	Range Rover	2014	premium unleaded (required)	510.0	8.0	AUTOMATIC	
9446	Chevrolet	Silverado 1500 Classic	2007	regular unleaded	295.0	8.0	AUTOMATIC	
3848	Hyundai	Elantra Touring	2011	regular unleaded	138.0	4.0	MANUAL	
4471	Ford	F-150	2015	flex-fuel (unleaded/E85)	282.0	6.0	AUTOMATIC	

Preprocessing

Missing Values

We'll preprocess the data first by checking for missing values and then removing them.

In [302…

```
data_car.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11914 entries, 0 to 11913
Data columns (total 16 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Make               11914 non-null  object
 1   Model              11914 non-null  object
 2   Year               11914 non-null  int64
 3   Engine Fuel Type   11911 non-null  object
 4   Engine HP          11845 non-null  float64
 5   Engine Cylinders   11884 non-null  float64
 6   Transmission Type  11914 non-null  object
 7   Driven_Wheels      11914 non-null  object
 8   Number of Doors    11908 non-null  float64
 9   Market Category    8172 non-null   object
 10  Vehicle Size       11914 non-null  object
 11  Vehicle Style      11914 non-null  object
 12  highway MPG        11914 non-null  int64
 13  city mpg           11914 non-null  int64
 14  Popularity         11914 non-null  int64
 15  MSRP               11914 non-null  int64
dtypes: float64(3), int64(5), object(8)
memory usage: 1.5+ MB
```

In [303…

```python
# Check for missing values in each column
data_car.isnull().sum()
#code that i have written end here
```

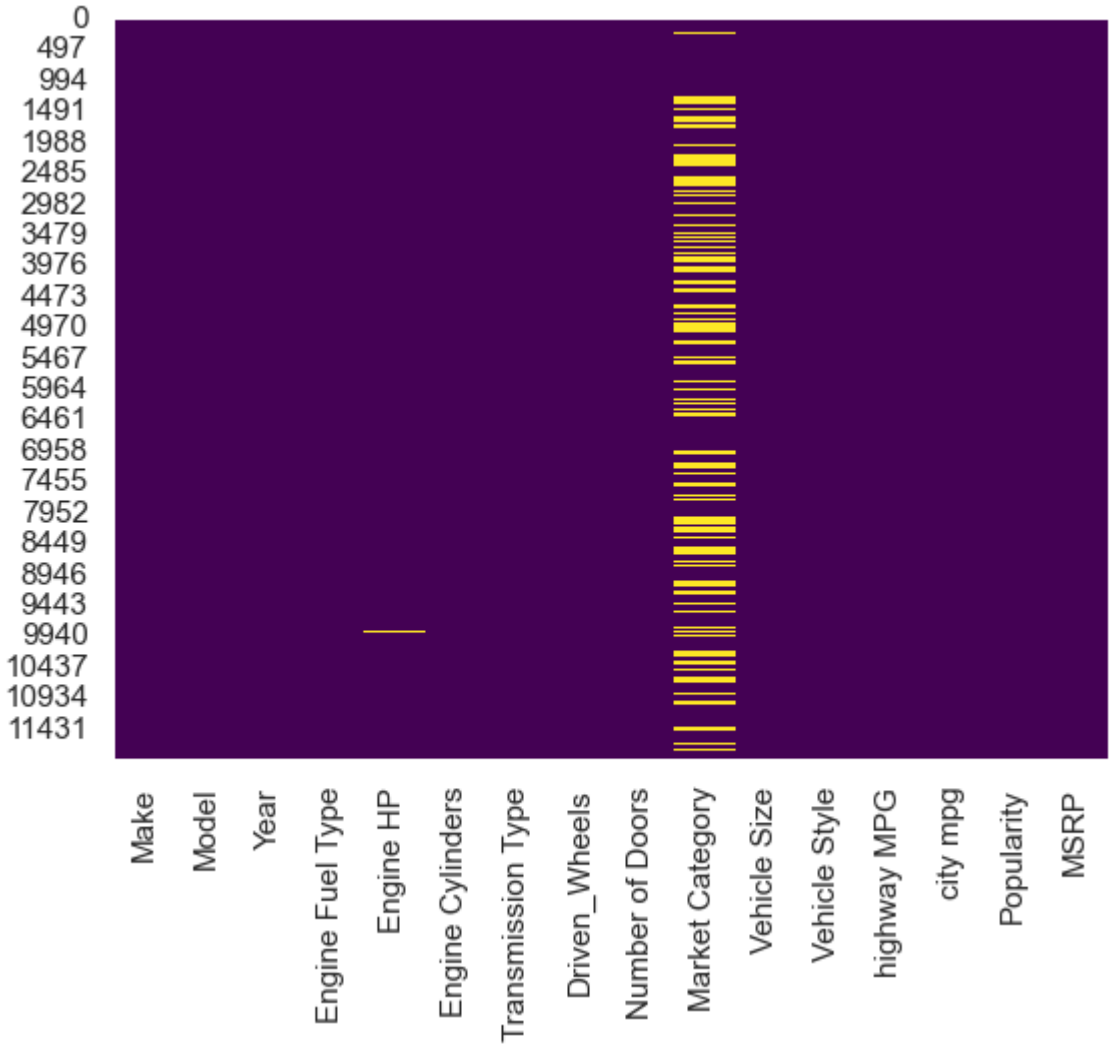
Out[303]:

```
Make                   0
Model                  0
Year                   0
Engine Fuel Type       3
Engine HP             69
Engine Cylinders      30
Transmission Type      0
Driven_Wheels          0
Number of Doors        6
Market Category     3742
Vehicle Size           0
Vehicle Style          0
highway MPG            0
city mpg               0
Popularity             0
MSRP                   0
dtype: int64
```

Heatmap as a visualization that shows the missing values.

In [304…

```python
# Visualize missing values using a heatmap
sns.heatmap(data_car.isnull(), cbar=False, cmap='viridis')
plt.show()
```



In [305…

```
#code that i have written
#to drop missing values
data_car = data_car.dropna()
data_car.isnull().sum()
```

Out[305]:

```
Make                 0
Model                0
Year                 0
Engine Fuel Type     0
Engine HP            0
Engine Cylinders     0
Transmission Type    0
Driven_Wheels        0
Number of Doors      0
Market Category      0
Vehicle Size         0
Vehicle Style        0
highway MPG          0
city mpg             0
Popularity           0
MSRP                 0
dtype: int64
```

Duplicated Values

After the missing values have been removed, we will look for duplicated data to make sure that the rows aren't duplicated that will ensure authenticity of the model and data integrity.

In [306…

```
#check for duplicated rows
num_duplicates = data_car.duplicated().sum()
print(f"Number of Duplicated Rows: {num_duplicates}")
```

```
Number of Duplicated Rows: 349
```

In [307…

```
df_no_duplicates = data_car.drop_duplicates(keep='first')
print("DataFrame without Duplicates:")
print(df_no_duplicates)
#code that i have written end here
```

```
DataFrame without Duplicates:
          Make       Model  Year                Engine Fuel Type  Engine HP  \
0          BMW  1 Series M  2011     premium unleaded (required)      335.0
1          BMW    1 Series  2011     premium unleaded (required)      300.0
2          BMW    1 Series  2011     premium unleaded (required)      300.0
3          BMW    1 Series  2011     premium unleaded (required)      230.0
4          BMW    1 Series  2011     premium unleaded (required)      230.0
...        ...         ...   ...                             ...        ...
11909    Acura         ZDX  2012     premium unleaded (required)      300.0
11910    Acura         ZDX  2012     premium unleaded (required)      300.0
11911    Acura         ZDX  2012     premium unleaded (required)      300.0
11912    Acura         ZDX  2013  premium unleaded (recommended)      300.0
11913  Lincoln      Zephyr  2006                regular unleaded      221.0

       Engine Cylinders Transmission Type      Driven_Wheels  Number of Doors  \
0                   6.0            MANUAL   rear wheel drive              2.0
1                   6.0            MANUAL   rear wheel drive              2.0
2                   6.0            MANUAL   rear wheel drive              2.0
3                   6.0            MANUAL   rear wheel drive              2.0
4                   6.0            MANUAL   rear wheel drive              2.0
...                 ...               ...                ...              ...
11909               6.0         AUTOMATIC    all wheel drive              4.0
11910               6.0         AUTOMATIC    all wheel drive              4.0
11911               6.0         AUTOMATIC    all wheel drive              4.0
11912               6.0         AUTOMATIC    all wheel drive              4.0
11913               6.0         AUTOMATIC  front wheel drive              4.0

                            Market Category Vehicle Size  Vehicle Style  \
0      Factory Tuner,Luxury,High-Performance      Compact          Coupe
1                        Luxury,Performance      Compact    Convertible
2                    Luxury,High-Performance      Compact          Coupe
3                        Luxury,Performance      Compact          Coupe
4                                   Luxury      Compact    Convertible
...                                    ...          ...            ...
11909             Crossover,Hatchback,Luxury      Midsize  4dr Hatchback
11910             Crossover,Hatchback,Luxury      Midsize  4dr Hatchback
11911             Crossover,Hatchback,Luxury      Midsize  4dr Hatchback
11912             Crossover,Hatchback,Luxury      Midsize  4dr Hatchback
11913                                Luxury      Midsize          Sedan

       highway MPG  city mpg  Popularity   MSRP
0               26        19        3916  46135
1               28        19        3916  40650
2               28        20        3916  36350
3               28        18        3916  29450
4               28        18        3916  34500
...            ...       ...         ...    ...
11909           23        16         204  46120
11910           23        16         204  56670
11911           23        16         204  50620
11912           23        16         204  50920
11913           26        17          61  28995

[7735 rows x 16 columns]
```

Exploratory Data Analysis (EDA) Analysis

Exploratory Data Analysis is a crucial phase in the data science workflow that involves examining and understanding the characteristics of the dataset. In the context of the automotive dataset chosen for this project, EDA plays a pivotal role in unraveling patterns, relationships, and insights that can inform subsequent modeling decisions, particularly in the application of linear regression.

Statistical Analysis

I have used the describe built in function to evaluate the count, mean standard deviation, minimum, the percentiles and the maximum values for each column.This helps to determine the range that most observations fall into, helps detect any outliers, and provides a greater knowledge of the distribution of the dataset. Using percentiles facilitates a more in-depth investigation of the dataset's dispersion, which is essential for making further modeling decisions.

In [308…

```
#code that i ahve written
#describe to get the stats
data_car.describe(percentiles = [0.25,0.50,0.75,0.85,0.90,1])
```

Out[308]:

	Year	Engine HP	Engine Cylinders	Number of Doors	highway MPG	city mpg	Po
count	8084.000000	8084.000000	8084.000000	8084.000000	8084.000000	8084.000000	8084
mean	2012.058634	274.642751	5.778080	3.414151	26.650915	19.553686	1499
std	6.317981	115.003475	1.886088	0.898087	7.720473	7.159964	1413
min	1990.000000	55.000000	0.000000	2.000000	12.000000	7.000000	2
25%	2010.000000	192.000000	4.000000	2.000000	22.000000	16.000000	549
50%	2015.000000	264.000000	6.000000	4.000000	26.000000	18.000000	1013
75%	2016.000000	320.000000	6.000000	4.000000	30.000000	22.000000	2009
85%	2017.000000	381.000000	8.000000	4.000000	33.000000	24.000000	2774
90%	2017.000000	430.000000	8.000000	4.000000	34.000000	26.000000	3916
100%	2017.000000	1001.000000	16.000000	4.000000	354.000000	137.000000	5657
max	2017.000000	1001.000000	16.000000	4.000000	354.000000	137.000000	5657

Bivariate Analysis: Numerical and Categorical Variables

In the bivariate analysis, I conducted a comprehensive examination of both numerical and categorical variables with respect to the target variable, 'Price.' For numerical variables such as 'Year,' 'Engine HP,' 'Engine Cylinders,' 'Number of Doors,' 'highway MPG,' 'city mpg,' and

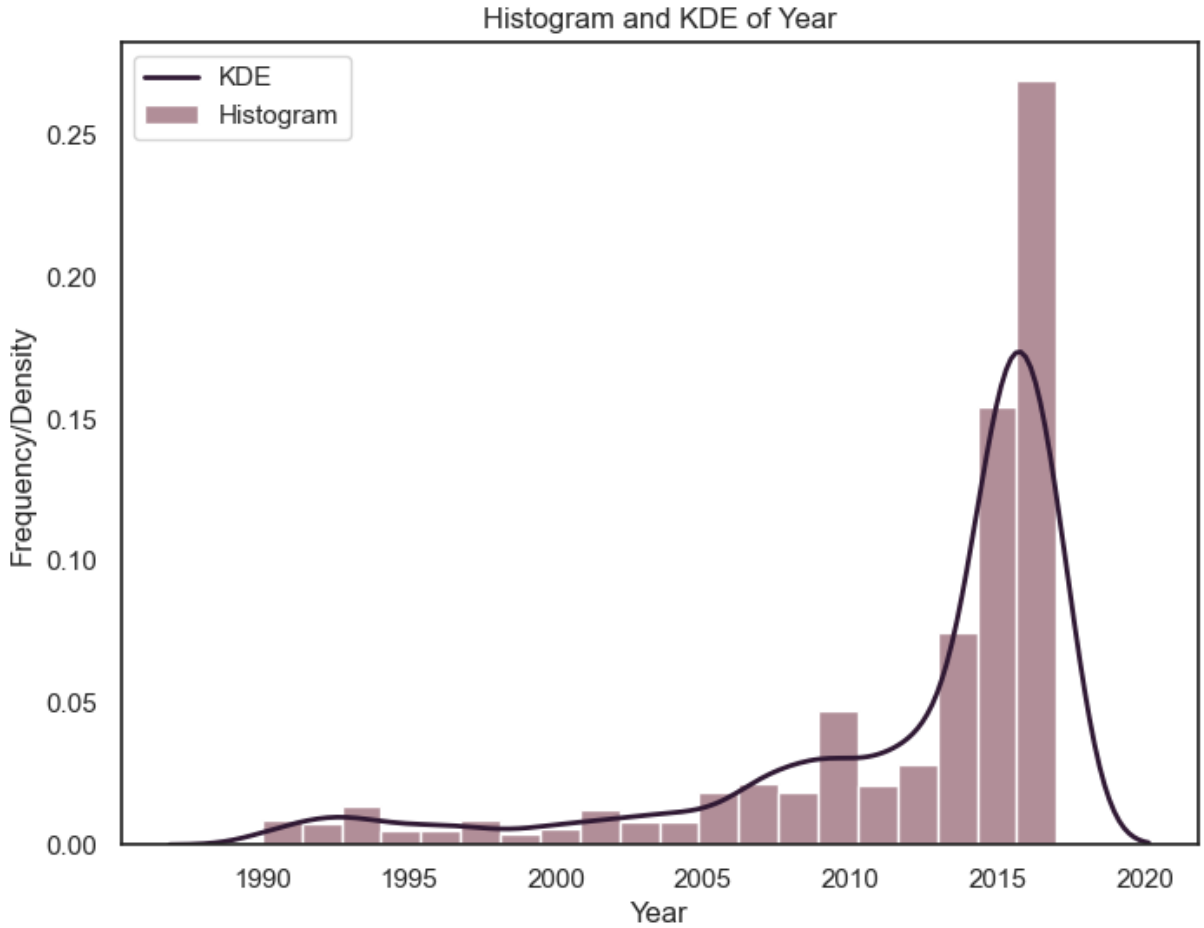
'Popularity,' I utilized various visualizations including scatter plots, histograms, and statistical summaries. These analyses aimed to uncover patterns, trends, and potential correlations between each numerical feature and the target variable
I started by first separating the numerical and categorical columns into separate list to analyze each column separately to get more insight into the data.

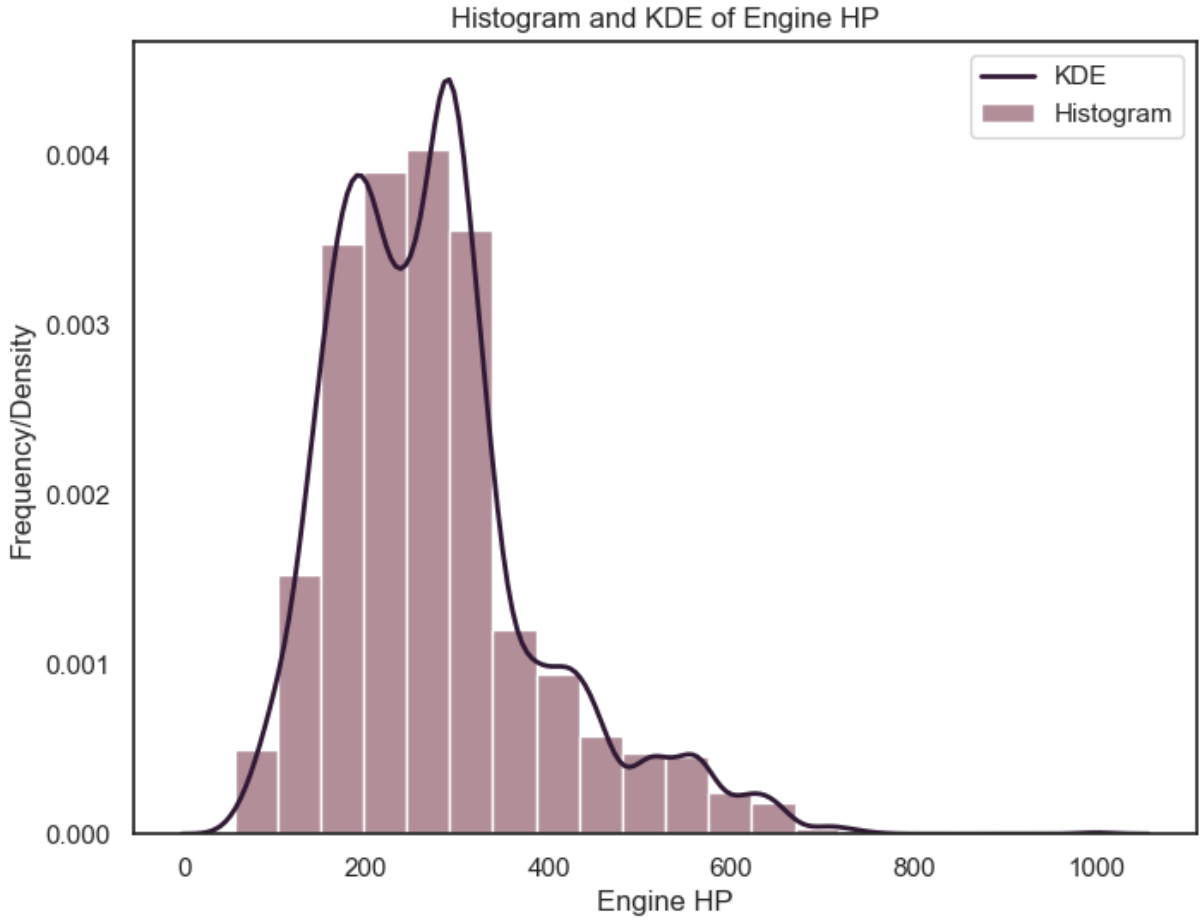
Numerical Columns

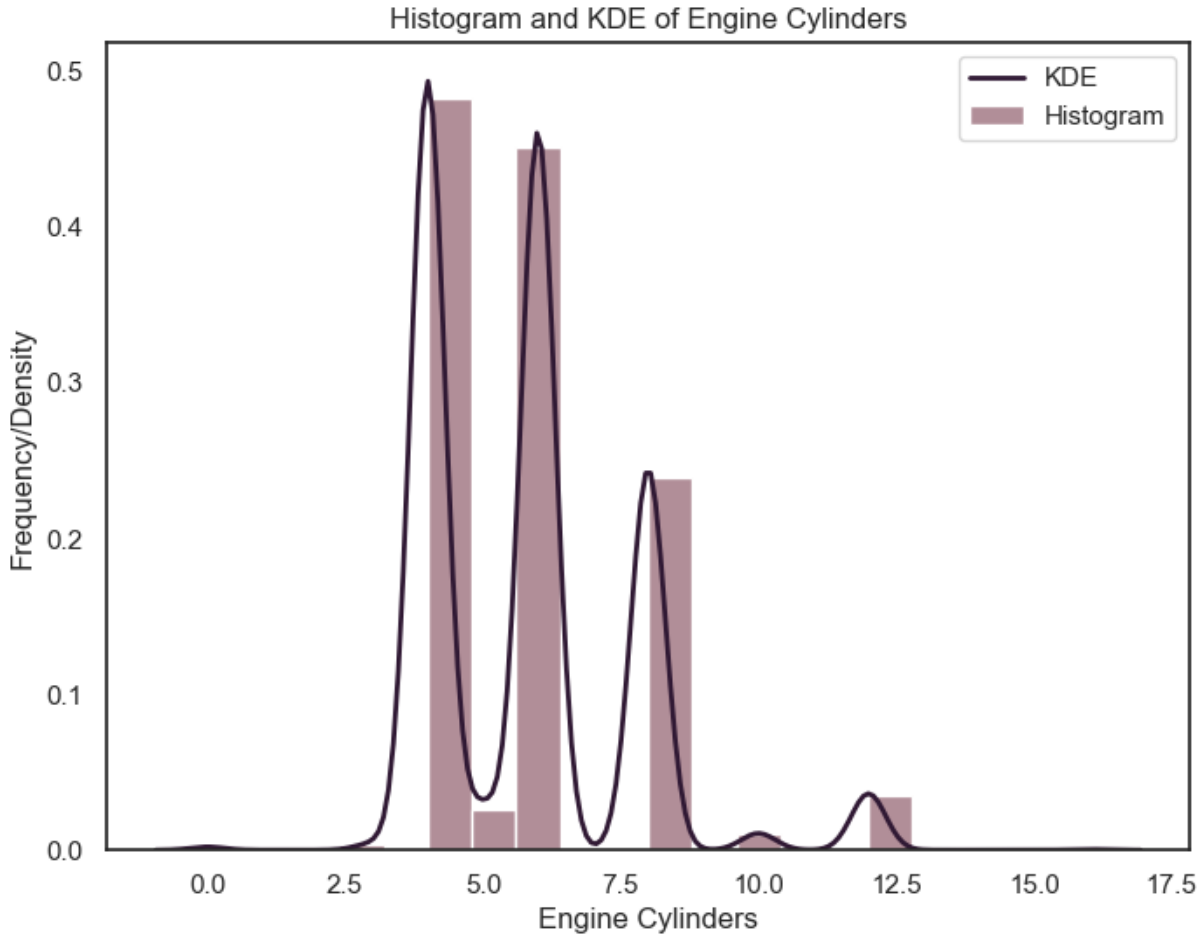
For the Numerical column, I used a for loop on it and plotted a histogram along with KDE(Kernel Density Estimate) plot that helps to give a better idea of each column. KDE plot make it easier to better visualize the trend shown in each graph...

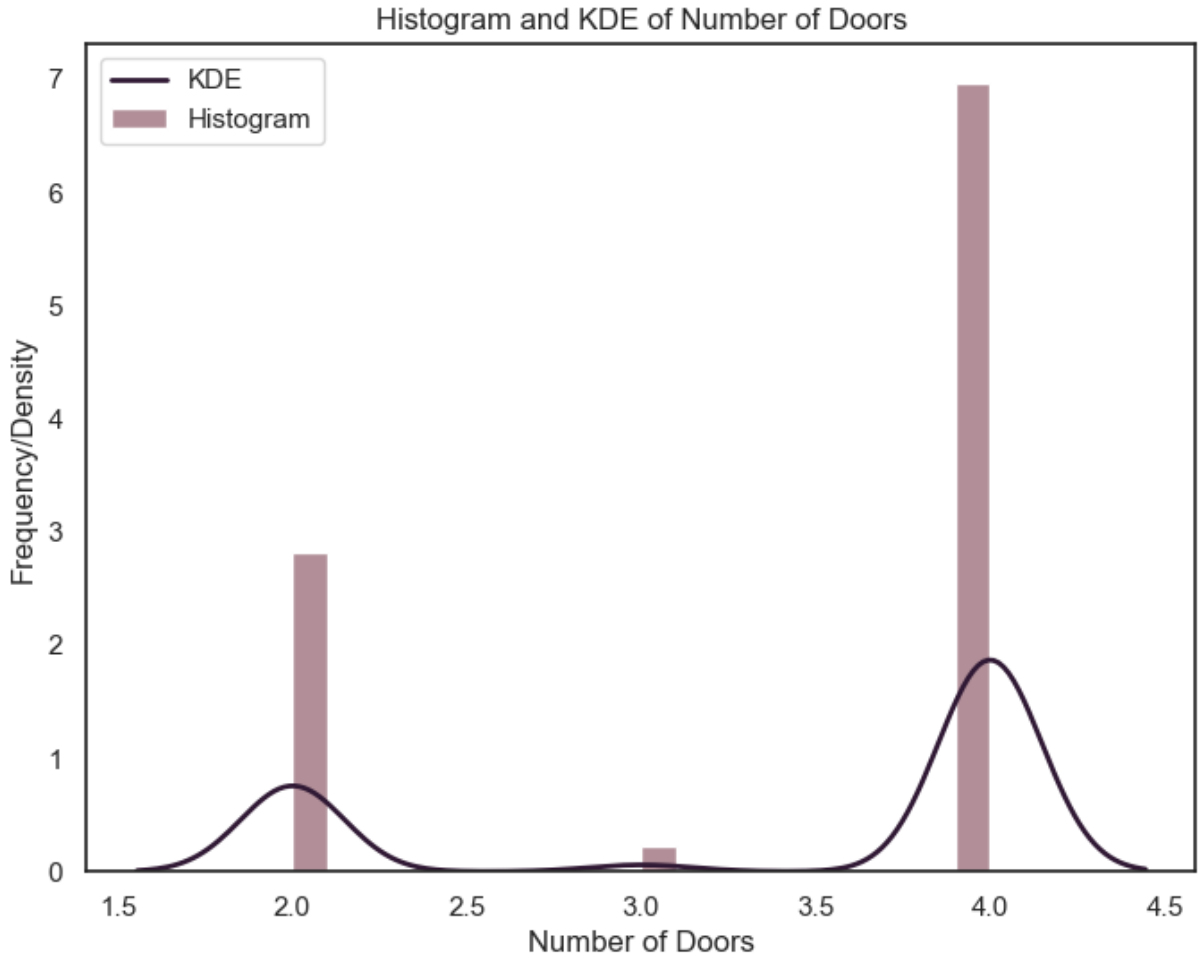
In [309…

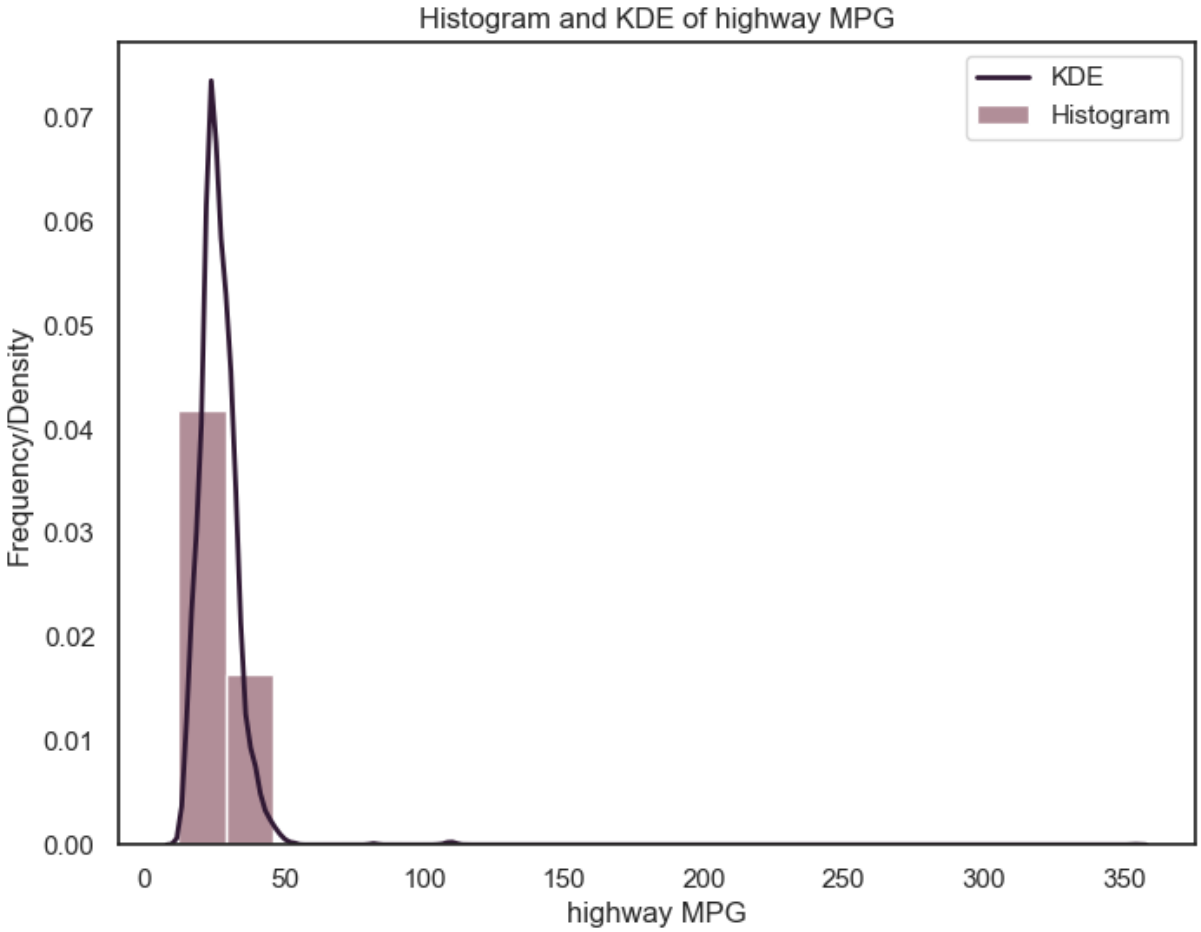
```
num_cols = ['Year', 'Engine HP', 'Engine Cylinders', 'Number of Doors', 'highway MP
#code that i have written ends here
for column in num_cols:
    plt.figure(figsize=(8, 6))
    # Plot histogram
    sns.histplot(data_car[column], bins=20, kde=False, color='#915F6D', alpha=0.7,
    # Plot KDE
    sns.kdeplot(data_car[column], color='#301934', linewidth=2, label='KDE')

    plt.title(f'Histogram and KDE of {column}')
    plt.xlabel(column)
    plt.ylabel('Frequency/Density')
    plt.legend()
    plt.show()
```

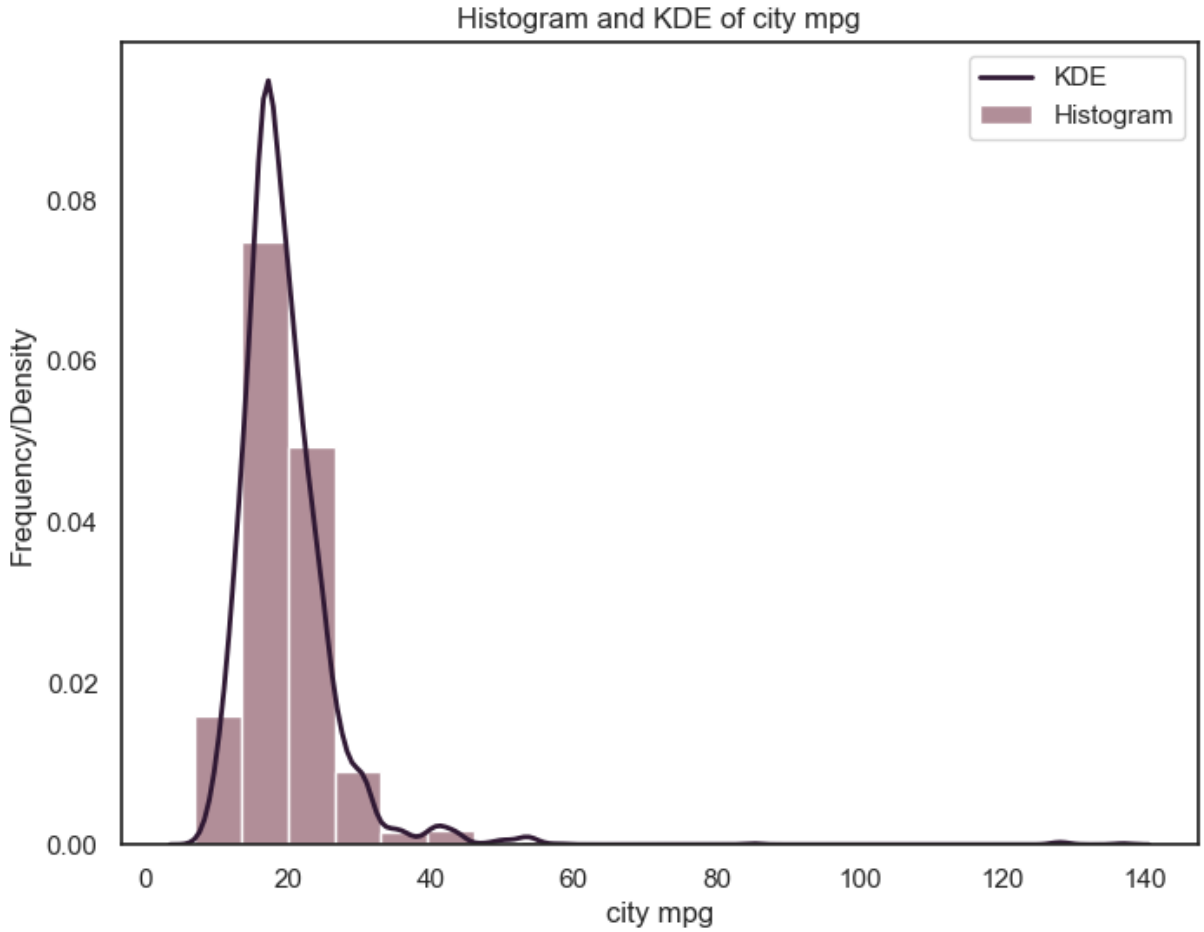

Histogram and KDE of Year
KDE
Histogram
Frequency/Density
0.25
0.20
0.15
0.10
0.05
0.00
1990
1995
2000
2005
2010
2015
2020
Year

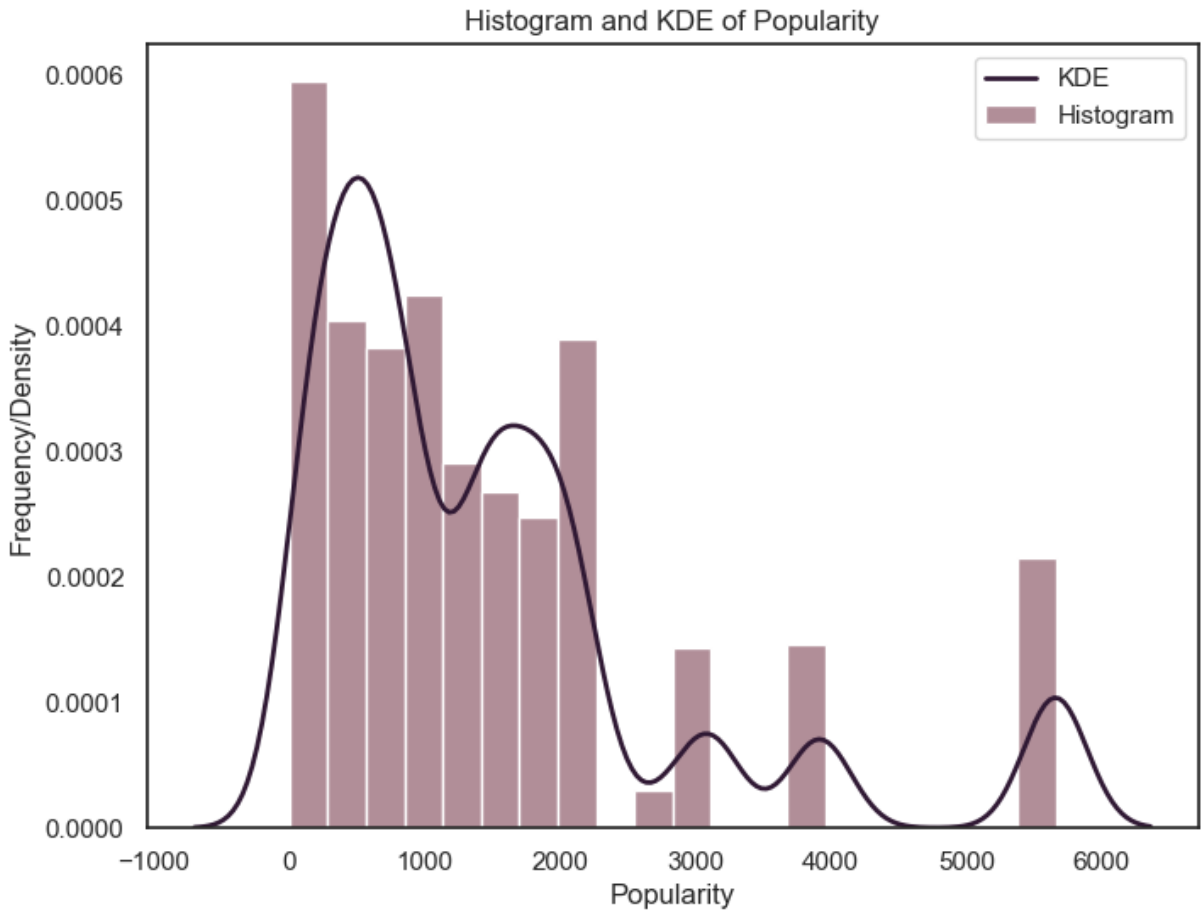

Histogram and KDE of Engine HP
KDE
Histogram
Frequency/Density
0.004
0.003
0.002
0.001
0.000
0
200
400
600
800
1000
Engine HP


Histogram and KDE of Engine Cylinders
KDE
Histogram
Frequency/Density
0.0
0.1
0.2
0.3
0.4
0.5
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Engine Cylinders


Histogram and KDE of Number of Doors
KDE
Histogram
Frequency/Density
0
1
2
3
4
5
6
7
1.5
2.0
2.5
3.0
3.5
4.0
4.5
Number of Doors


Histogram and KDE of highway MPG
KDE
Histogram
Frequency/Density
0.07
0.06
0.05
0.04
0.03
0.02
0.01
0.00
0
50
100
150
200
250
300
350
highway MPG


Histogram and KDE of city mpg
KDE
Histogram
Frequency/Density
0.00
0.02
0.04
0.06
0.08
0
20
40
60
80
100
120
140
city mpg

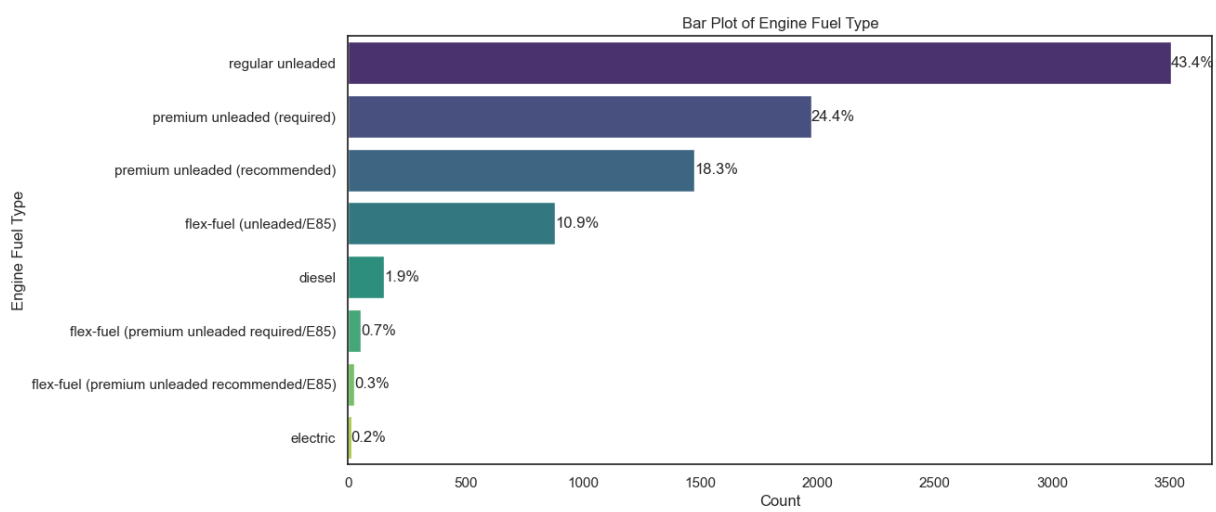

Histogram and KDE of Popularity
KDE
Histogram
Frequency/Density
0.0000
0.0001
0.0002
0.0003
0.0004
0.0005
0.0006
−1000
0
1000
2000
3000
4000
5000
6000
Popularity

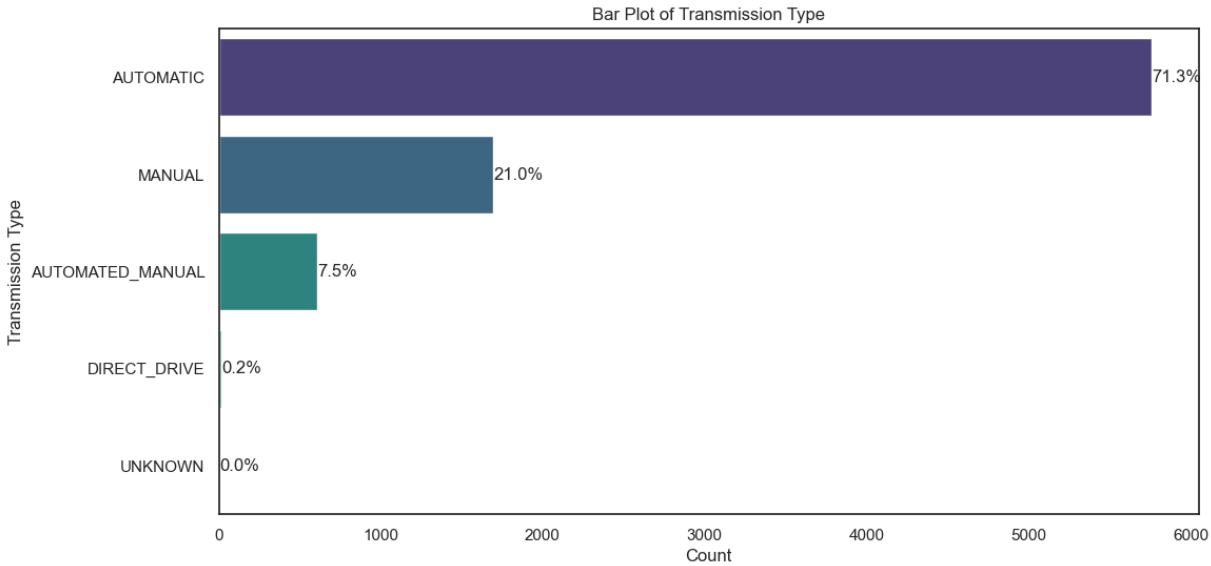
Categorical Columns

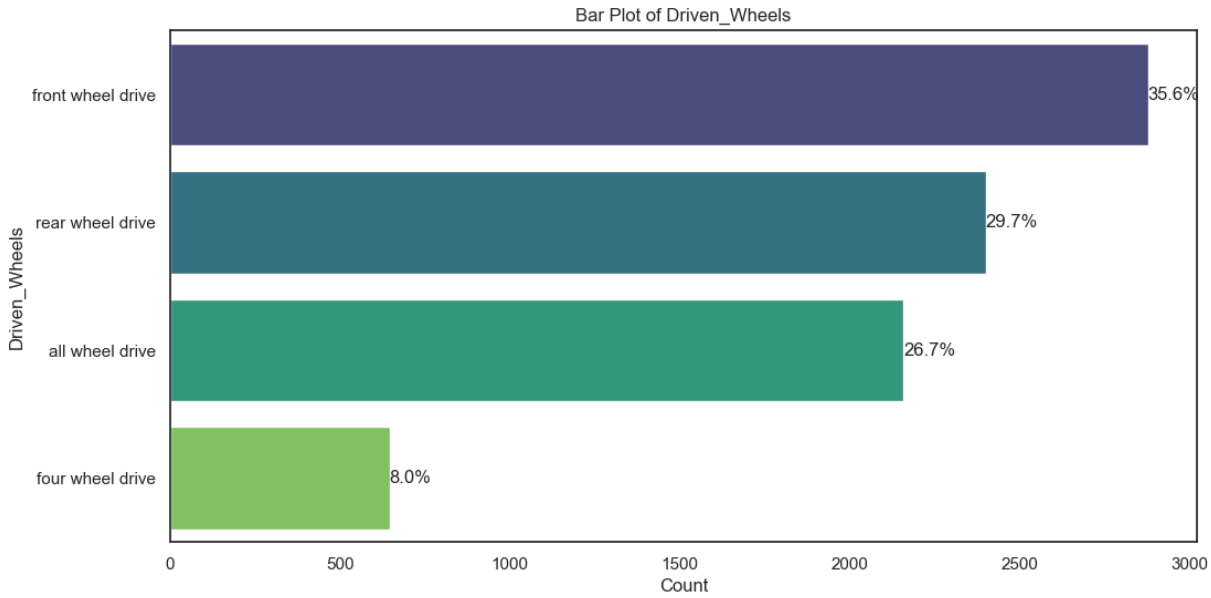
Likewise, I created seperate graph first to visualize the trends for categorical columns. Upon making graphs, I realized that the 'Make' and Market Category column has a vast data so I visualized them seperately. I plotted bar graph for the categorical columns along with the percentages.

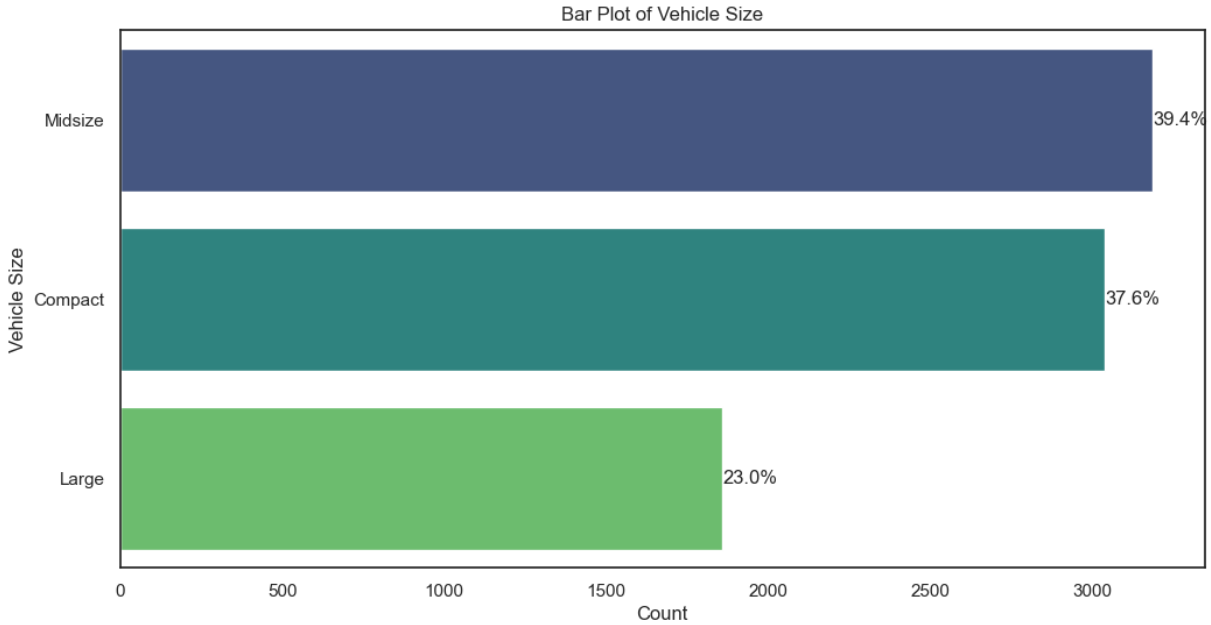
In [310…

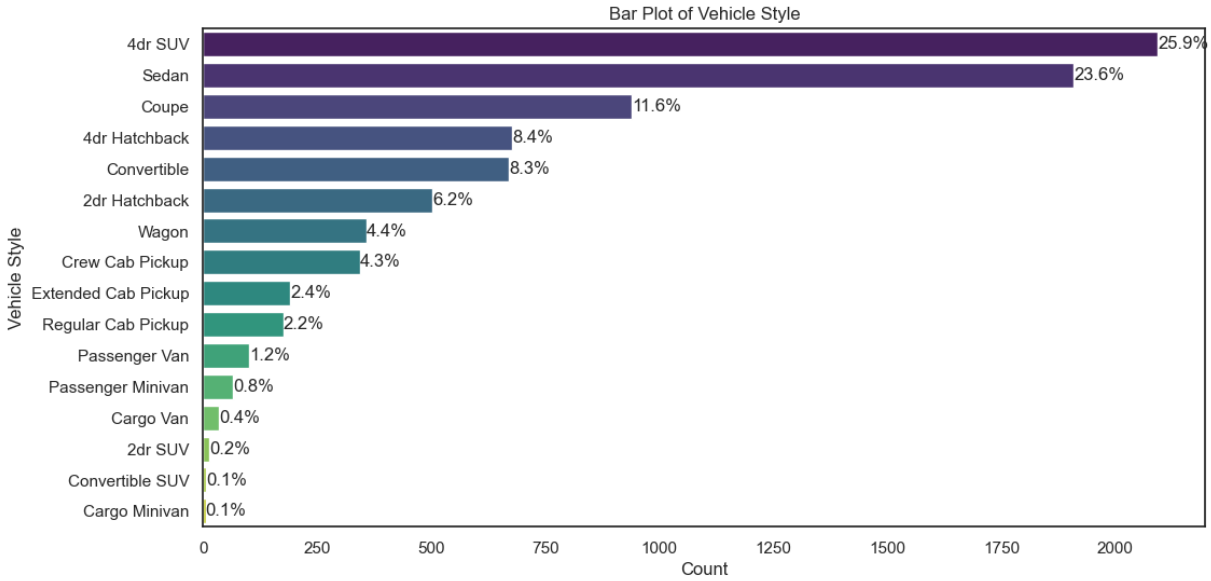
```
#code that i ahve written
cat_cols = ['Engine Fuel Type', 'Transmission Type', 'Driven_Wheels', 'Vehicle Size

for column in cat_cols:
    plt.figure(figsize=(12, 6))

    # Plot bar plot
    sns.barplot(y=data_car[column].value_counts().index, x=df[column].value_counts(
#code that i ahve written ends here
    # Add percentage labels
    total_count = len(data_car[column])
    for p in plt.gca().patches:
        percentage = '{:.1f}%'.format(100 * p.get_width() / total_count)
        x = p.get_x() + p.get_width() + 0.02
        y = p.get_y() + p.get_height() / 2
        plt.annotate(percentage, (x, y), ha='left', va='center')

    plt.title(f'Bar Plot of {column}')
    plt.xlabel('Count')
    plt.show()
```




Bar Plot of Transmission Type
Transmission Type
AUTOMATIC
MANUAL
AUTOMATED_MANUAL
DIRECT_DRIVE
UNKNOWN
71.3%
21.0%
7.5%
0.2%
0.0%
0
1000
2000
3000
4000
5000
6000
Count


Bar Plot of Driven_Wheels
Driven_Wheels
front wheel drive
rear wheel drive
all wheel drive
four wheel drive
35.6%
29.7%
26.7%
8.0%
0
500
1000
1500
2000
2500
3000
Count

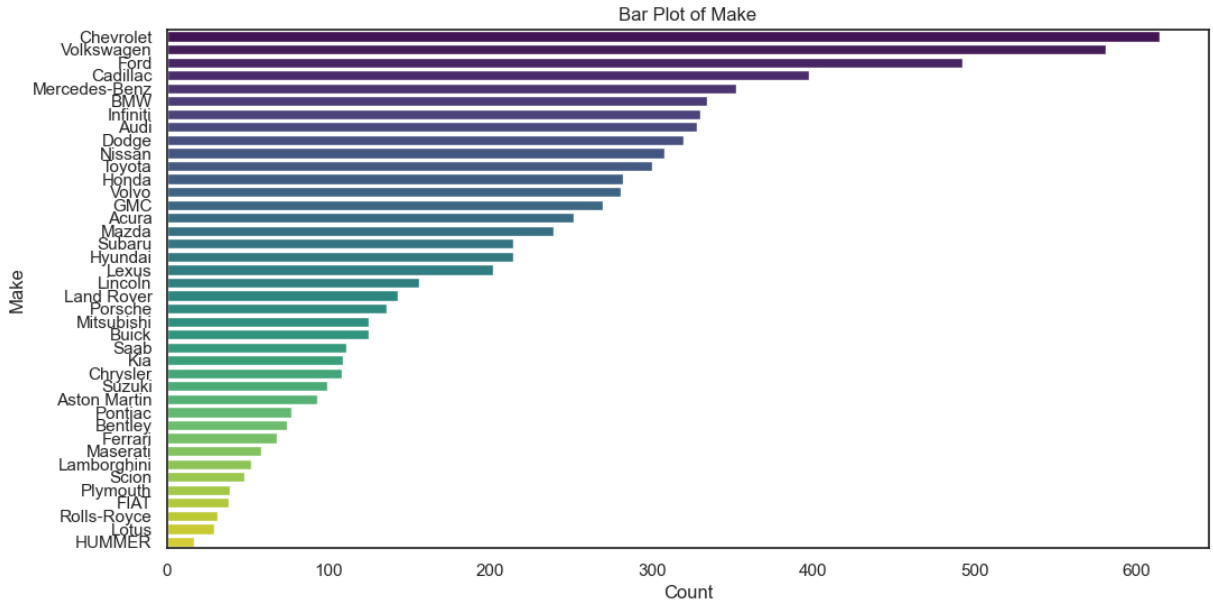

Bar Plot of Vehicle Size
Vehicle Size
Midsize
Compact
Large
39.4%
37.6%
23.0%
0
500
1000
1500
2000
2500
3000
Count

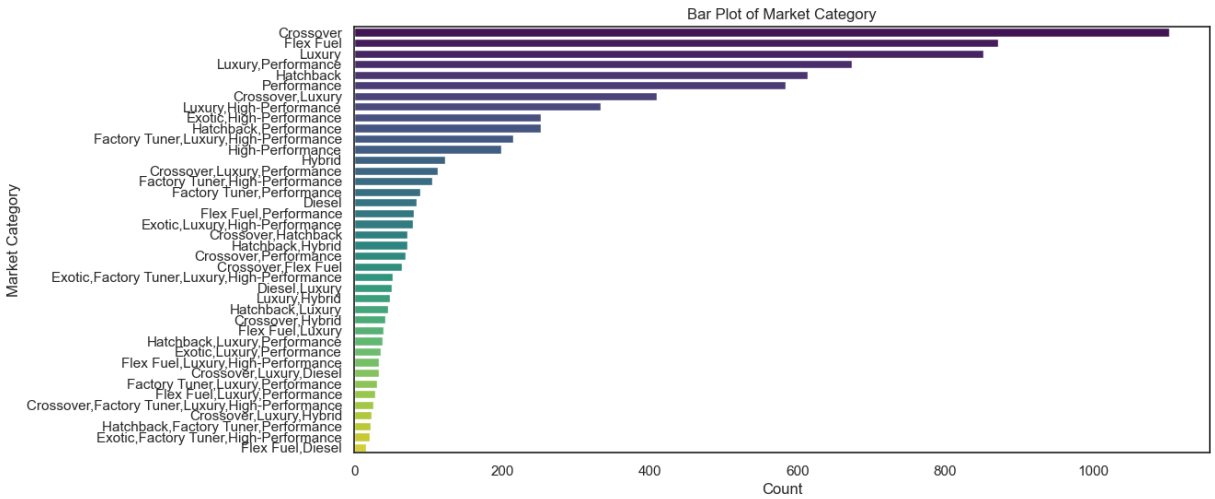


Because the data was so huge to be plotted in the form of graph,as shown below, I chose a certain length for the graphs to be displayed. In this, I chose to display about 40 categories from both columns which helped me to view the trend without being so congested.

In [311…

```
#code that i have written
sel_cat_len = 40  # Specify the desired number of categories to display

# Plot for 'Make'
plt.figure(figsize=(12, 6))
sns.barplot(y=data_car['Make'].value_counts().index[:sel_cat_len],
            x=data_car['Make'].value_counts()[:sel_cat_len], palette='viridis')
plt.title(f'Bar Plot of Make')
plt.xlabel('Count')
plt.show()

# Plot for 'Market Category'
plt.figure(figsize=(12, 6))
sns.barplot(y=data_car['Market Category'].value_counts().index[:sel_cat_len],
            x=data_car['Market Category'].value_counts()[:sel_cat_len], palette='vi
plt.title(f'Bar Plot of Market Category')
plt.xlabel('Count')
plt.show()
```





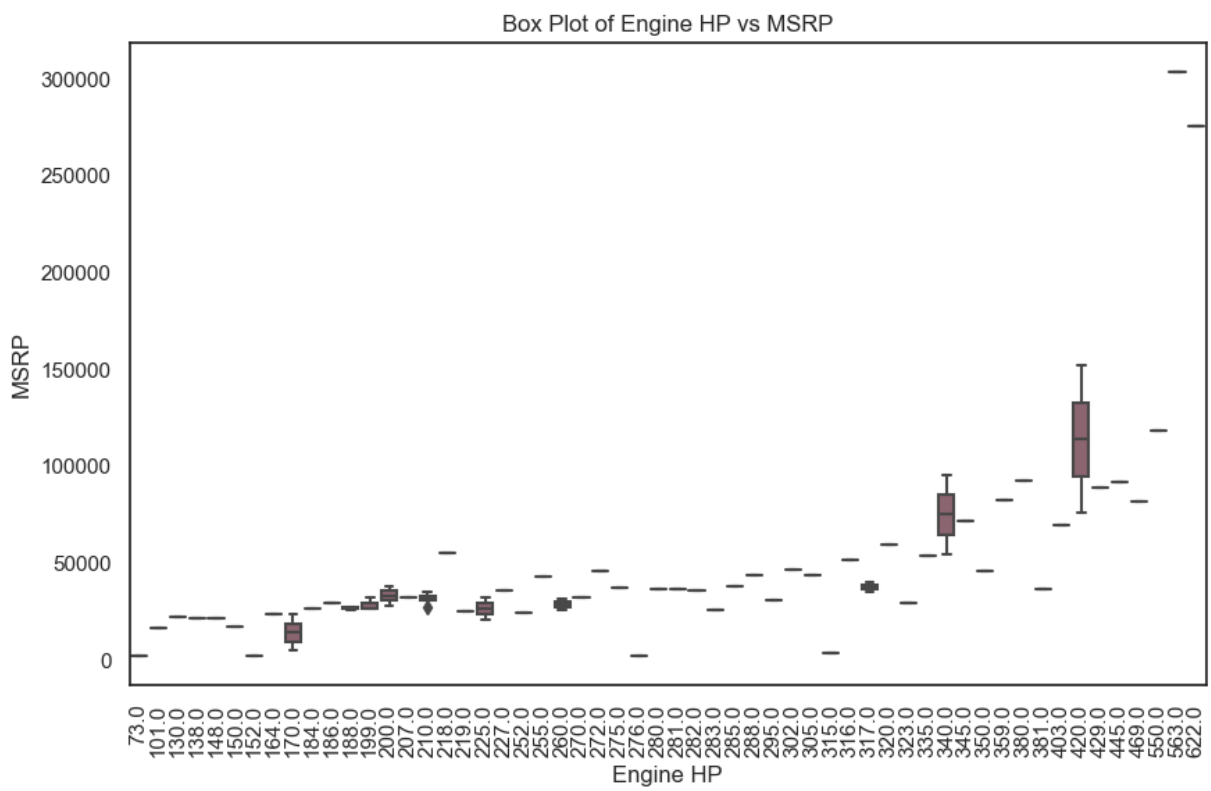
Numerical Columns vs MSRP

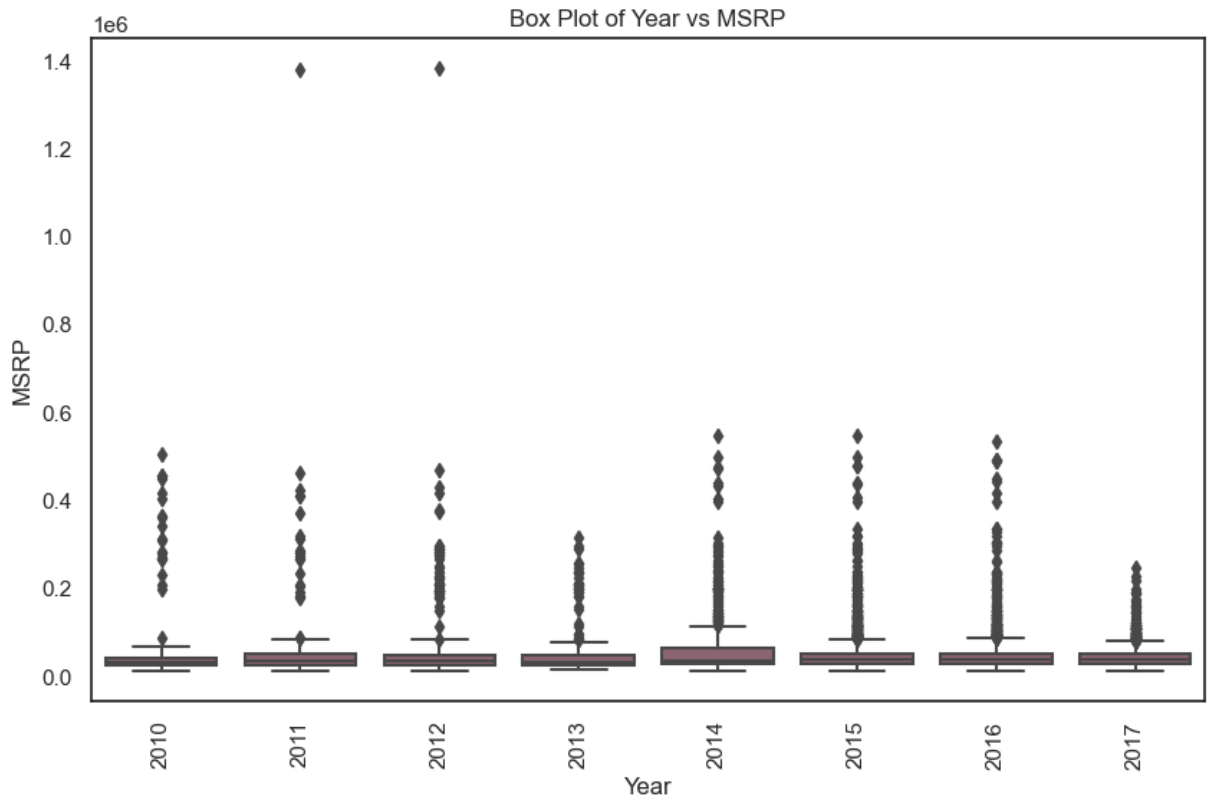
After understanding the trends in the graph, I first analyzed the numerical columns and which factor affected the MSRP (Manufacturer's Suggested Retail Price) as it was the target variable in my machine-learning model. By pinpointing the factors that play a role in determining MSRP, this initial exploration serves as a crucial stepping stone for the subsequent development of our machine-learning model.
70 random values were sampled for the 'Engine HP' column, offering a manageable yet representative look at the distribution of engine horsepower and its relationship to MSRP. To streamline the dataset for a more focused analysis, the 'Year' column was also limited to entries from 2010 forward. This deliberate method guarantees a more nuanced analysis of the selected variables in respect to MSRP while also speeding up the visualization process.

In [312…

```
sel_hp_values = data_car['Engine HP'].sample(n=70, random_state=42)
sel_years = data_car[data_car['Year'] >= 2010]['Year']

# Box plot for 'Engine HP'
plt.figure(figsize=(10, 6))
```

```
sns.boxplot(x=sel_hp_values, y=data_car['MSRP'], color='#915F6D')

# Rotate x-axis labels for better visibility
plt.tick_params(axis='x', rotation=90)

plt.title('Box Plot of Engine HP vs MSRP')
plt.xlabel('Engine HP')
plt.ylabel('MSRP')
plt.show()

# Box plot for 'Year'
plt.figure(figsize=(10, 6))
sns.boxplot(x=sel_years, y=data_car['MSRP'], color='#915F6D')

# Rotate x-axis labels for better visibility
plt.tick_params(axis='x', rotation=90)

plt.title('Box Plot of Year vs MSRP')
plt.xlabel('Year')
plt.ylabel('MSRP')
plt.show()
```

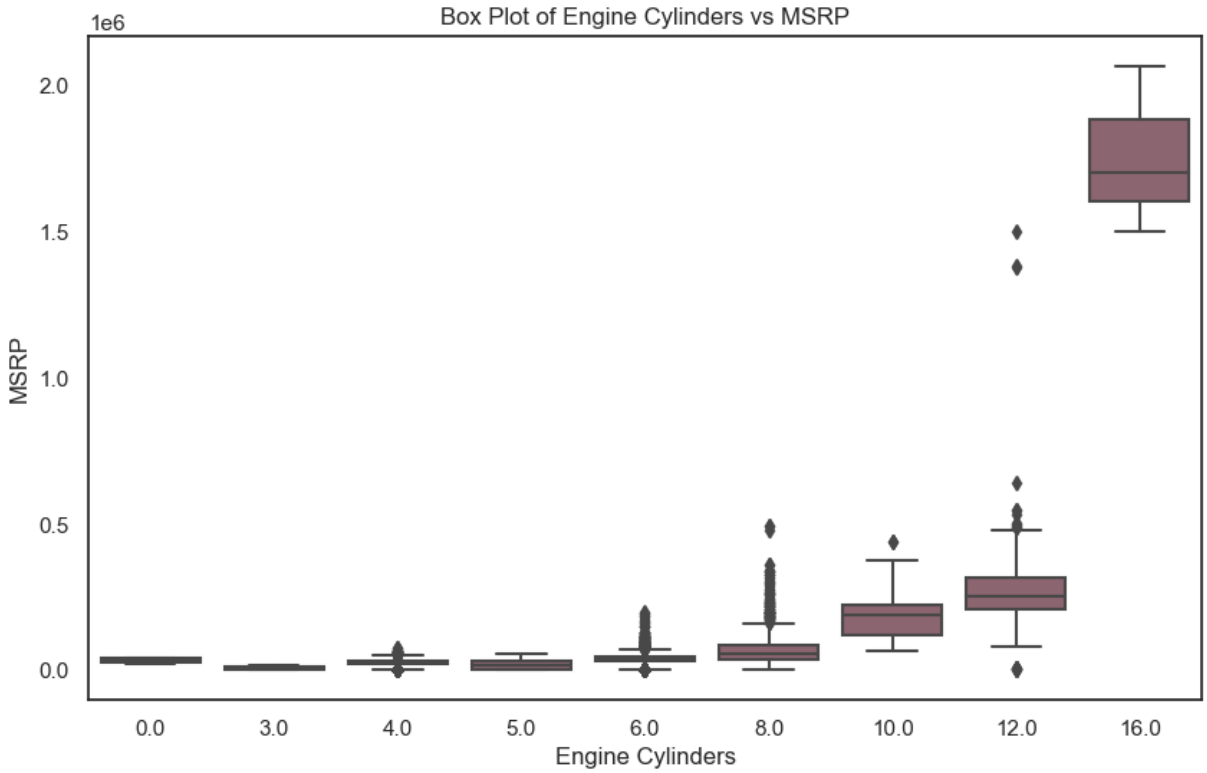


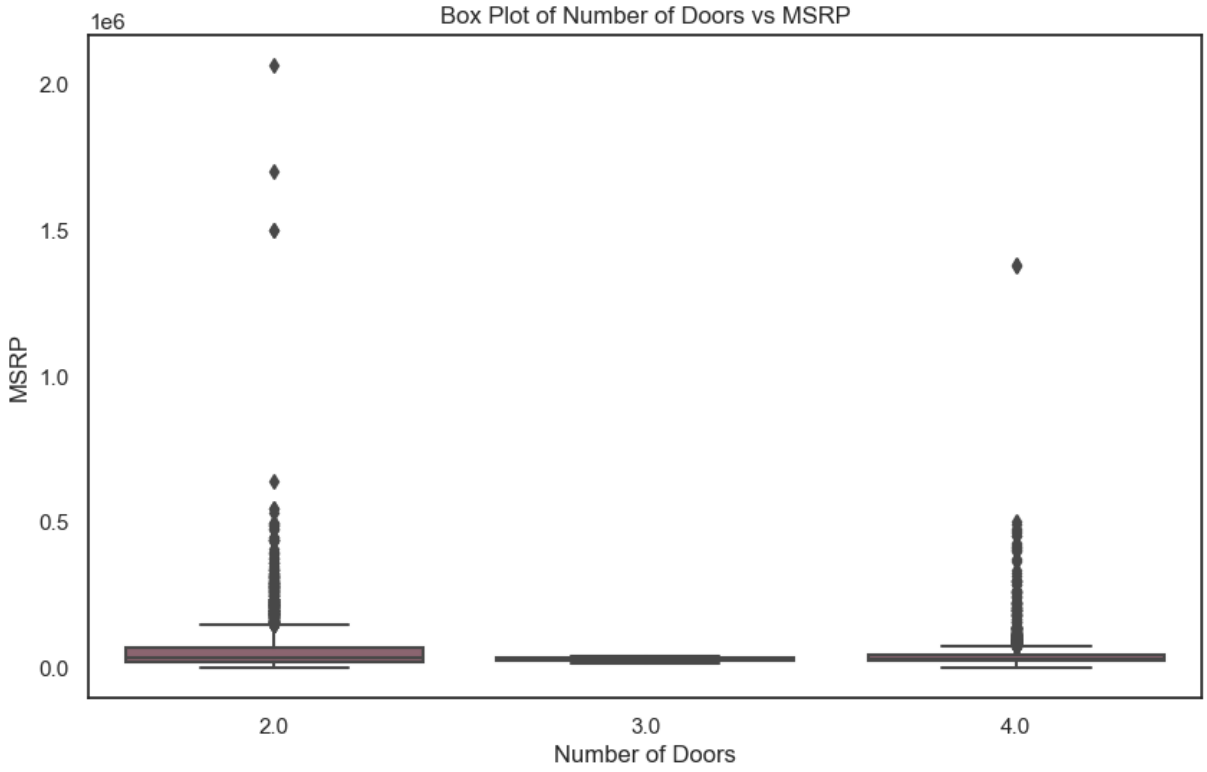


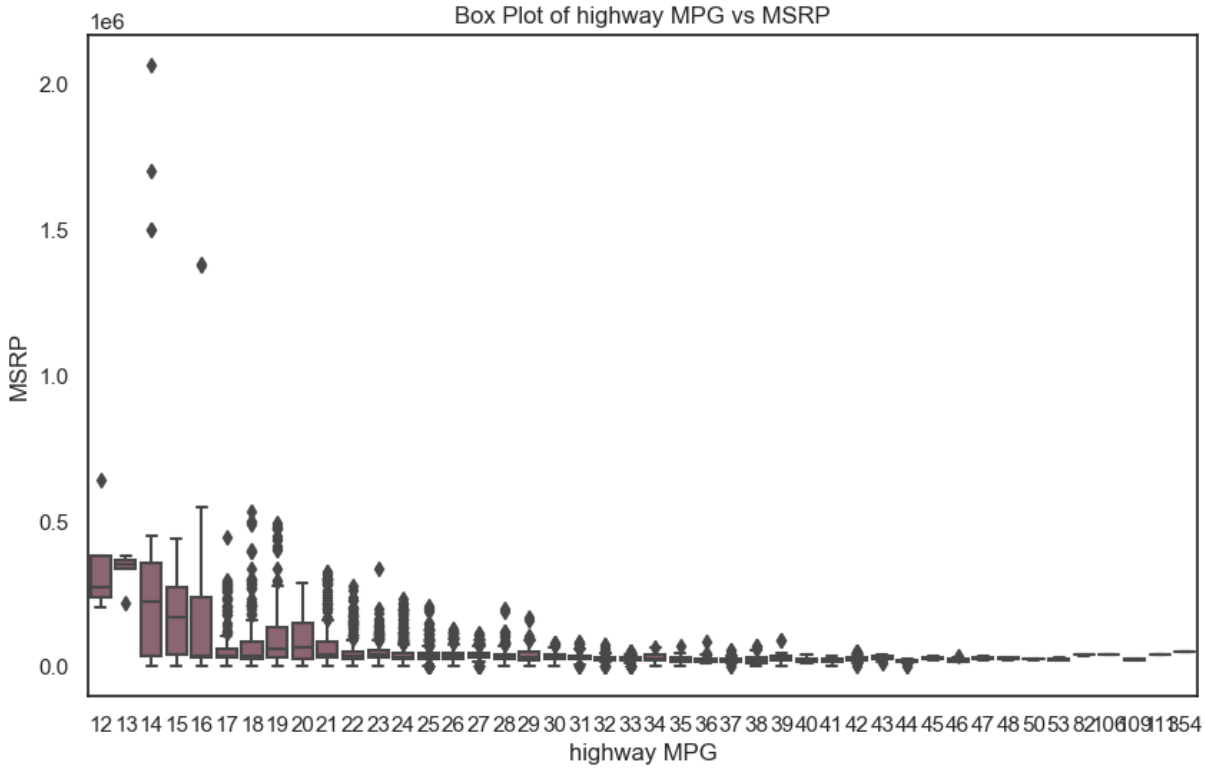
The visualization for the remaining column for numerical columns.

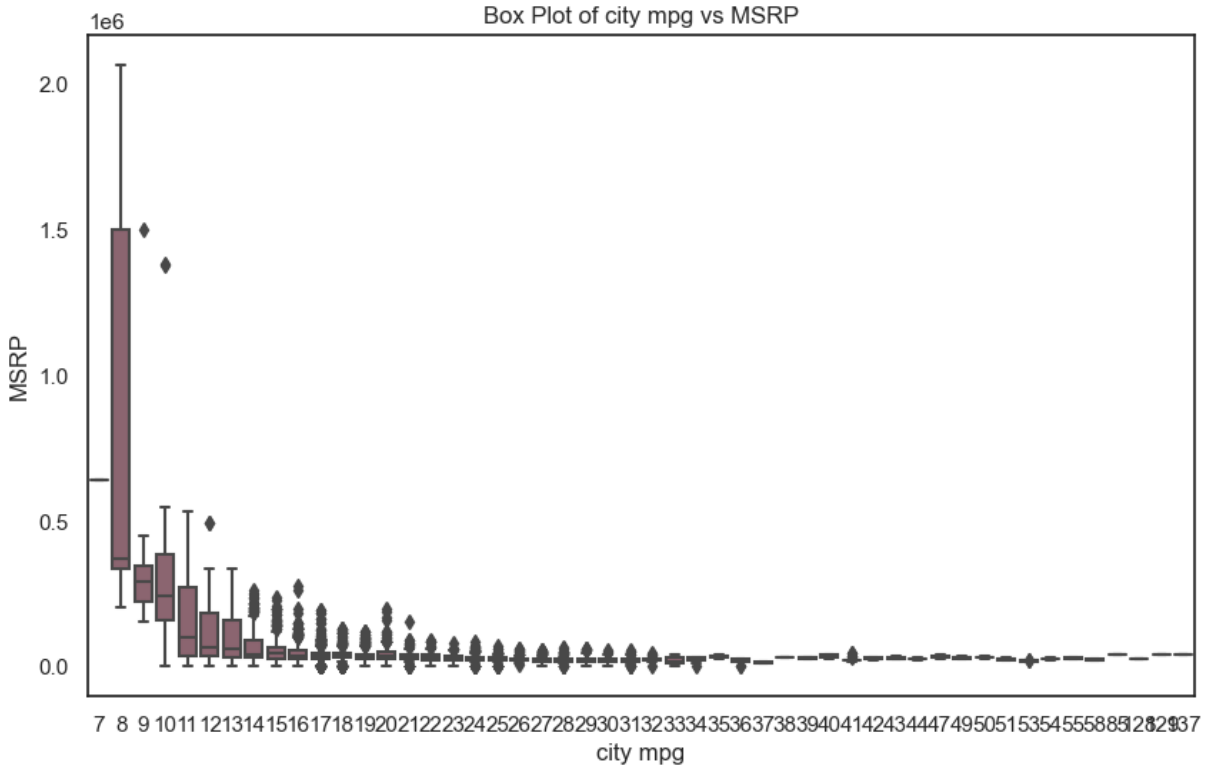
In [313…

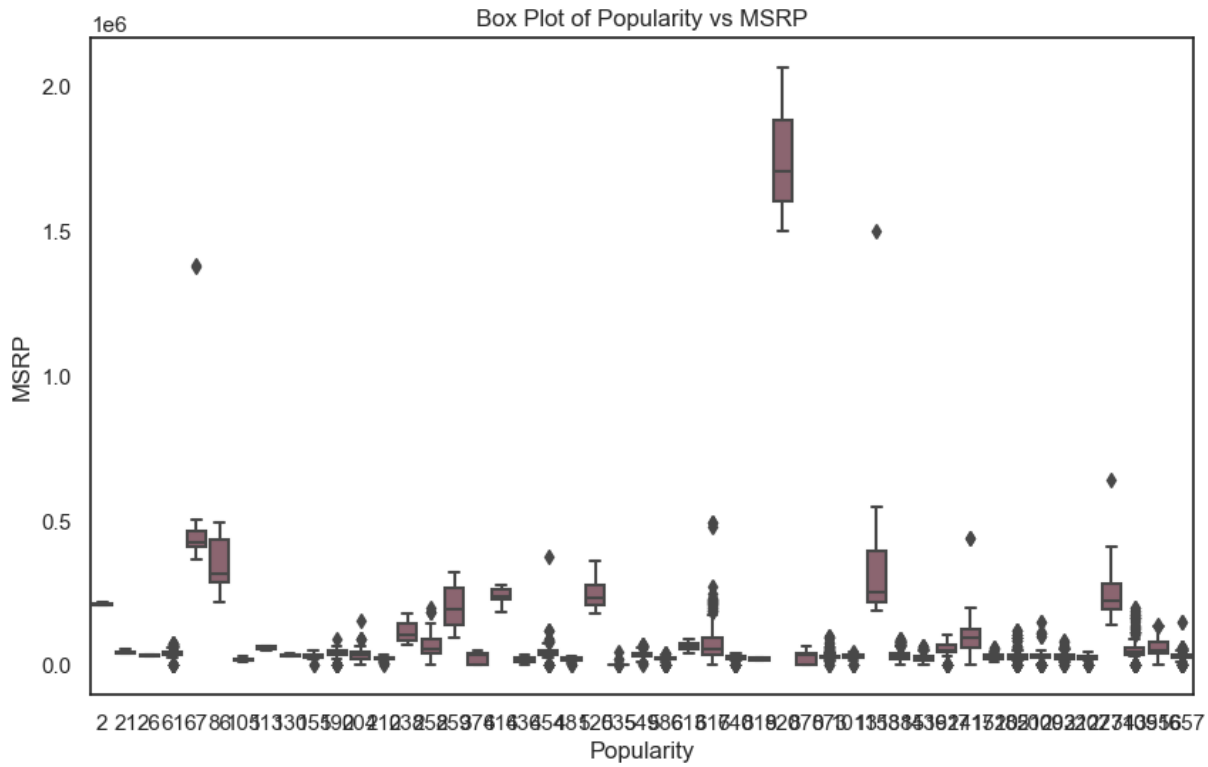
```
num_cols = ['Engine Cylinders', 'Number of Doors', 'highway MPG', 'city mpg', 'Popu

for column in num_cols:
    plt.figure(figsize=(10, 6))

    # Plot box plot
    sns.boxplot(x=data_car[column], y=data_car['MSRP'], color='#915F6D')

    plt.title(f'Box Plot of {column} vs MSRP')
    plt.xlabel(column)
    plt.ylabel('MSRP')
    plt.show()
```


Box Plot of Engine Cylinders vs MSRP
1e6
2.0
1.5
1.0
0.5
0.0
MSRP
0.0
3.0
4.0
5.0
6.0
8.0
10.0
12.0
16.0
Engine Cylinders


Box Plot of Number of Doors vs MSRP
1e6
2.0
1.5
1.0
0.5
0.0
MSRP
2.0
3.0
4.0
Number of Doors


Box Plot of highway MPG vs MSRP
1e6
2.0
1.5
1.0
0.5
0.0
MSRP
12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 50 53 82 106 109 113 354
highway MPG


Box Plot of city mpg vs MSRP
1e6
2.0
1.5
1.0
0.5
0.0
MSRP
7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 47 49 50 51 53 54 55 58 85 128 129 137
city mpg

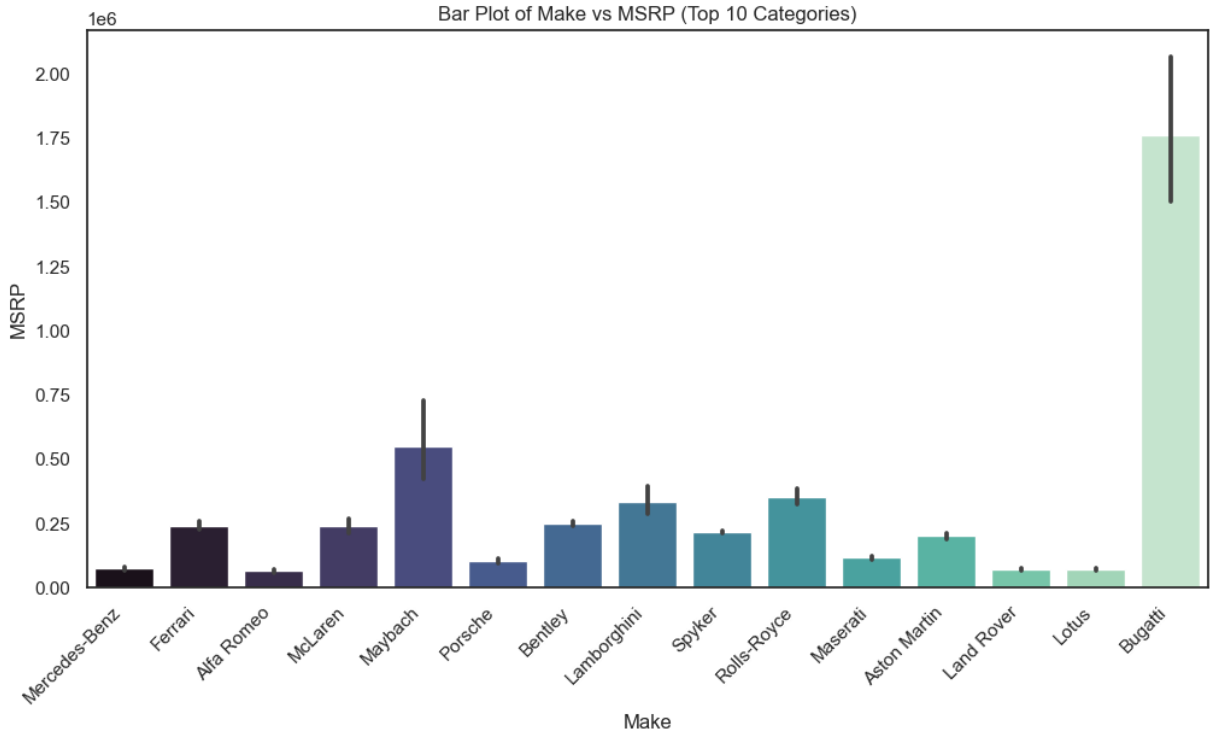


Categorical Columns vs MSRP

Likewise, I have plotted graphs for Categorical columns vs the MSRP to have better sense of the features in relation to the MSRP to have more targeted analysis. I have chosen the top data for the categorical columns where required as the data was extremely huge to be visualized in a graph so as for the 'Make' column I chose the top 15 Make and plotted it against the MSRP.

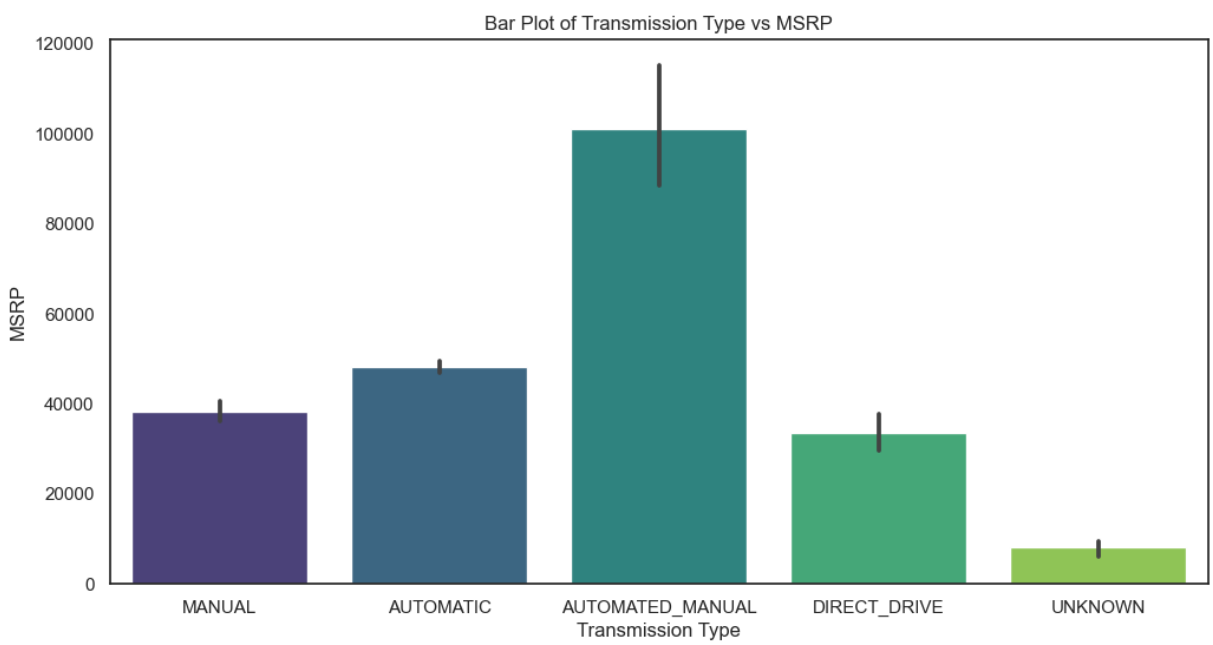
In [314…

```
top_n_cat = data_car.groupby('Make')['MSRP'].mean().sort_values(ascending=False).he

# Filter the DataFrame for the top 15 categories
df_top_n_cat = data_car[data_car['Make'].isin(top_n_cat)]

# Bar Plot
plt.figure(figsize=(12, 6))
sns.barplot(x='Make', y='MSRP', data=df_top_n_cat, palette='mako')
plt.title('Bar Plot of Make vs MSRP (Top 10 Categories)')
plt.xlabel('Make')
plt.ylabel('MSRP')
plt.xticks(rotation=45, ha='right')
plt.show()
```



In [315…

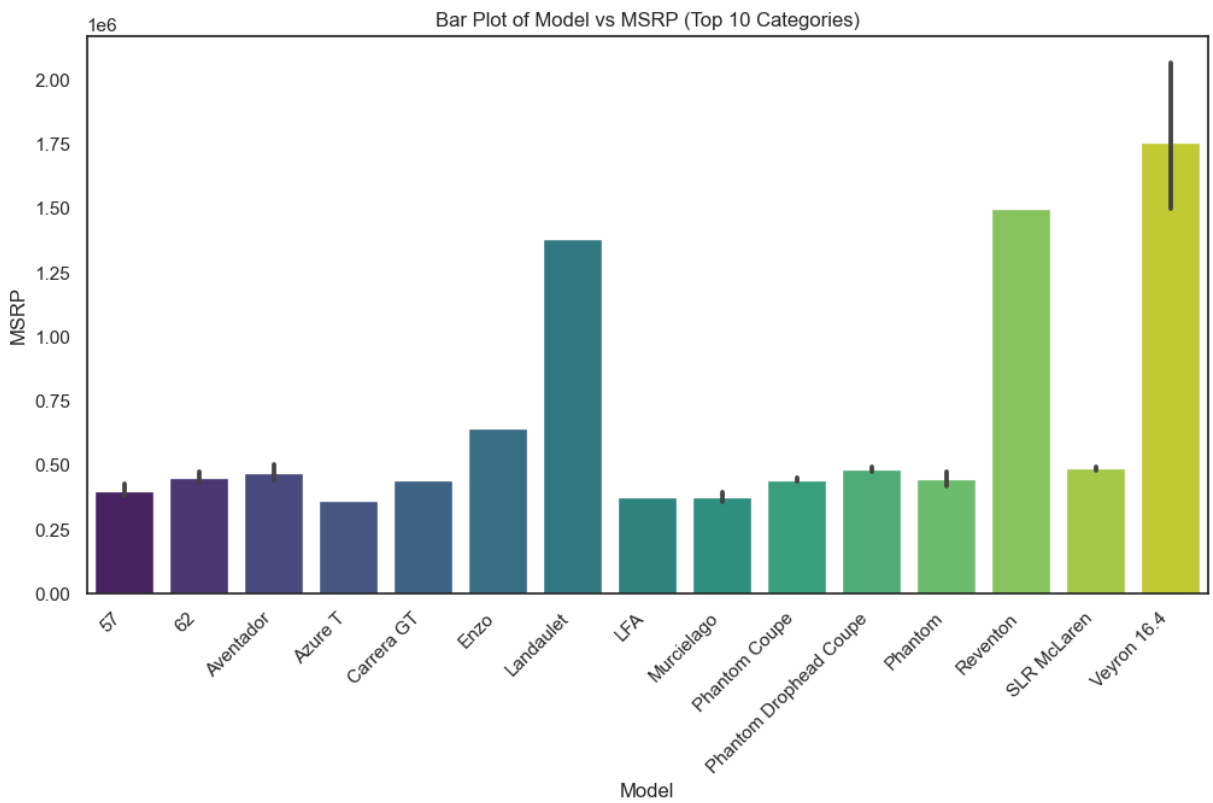
```
plt.figure(figsize=(12, 6))
sns.barplot(x='Transmission Type', y='MSRP', data=data_car, palette='viridis')
plt.title('Bar Plot of Transmission Type vs MSRP')
plt.xlabel('Transmission Type')
plt.ylabel('MSRP')
plt.show()
#code that i have written ends here
```

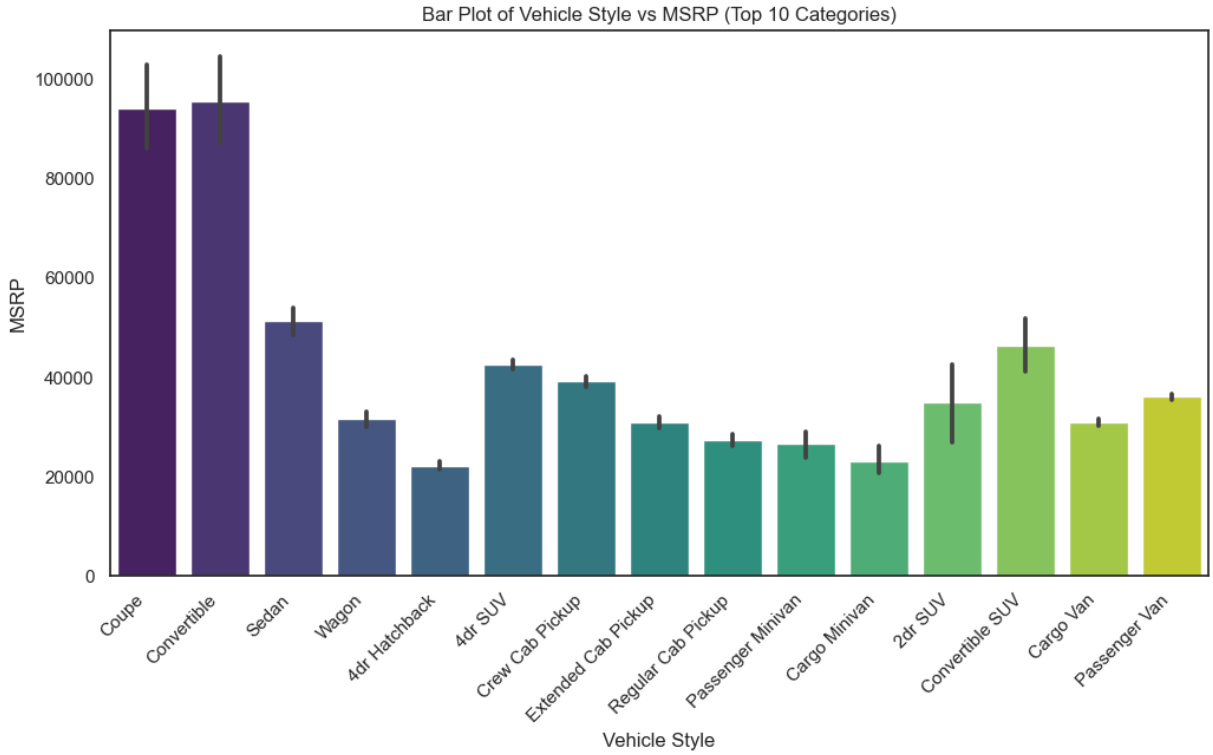


In [316…

```
# Select the top 15 categories based on average MSRP for 'Model'
top_model_cat = data_car.groupby('Model')['MSRP'].mean().sort_values(ascending=Fals
df_top_model_cat = data_car[data_car['Model'].isin(top_model_cat)]

#code that i have written
```

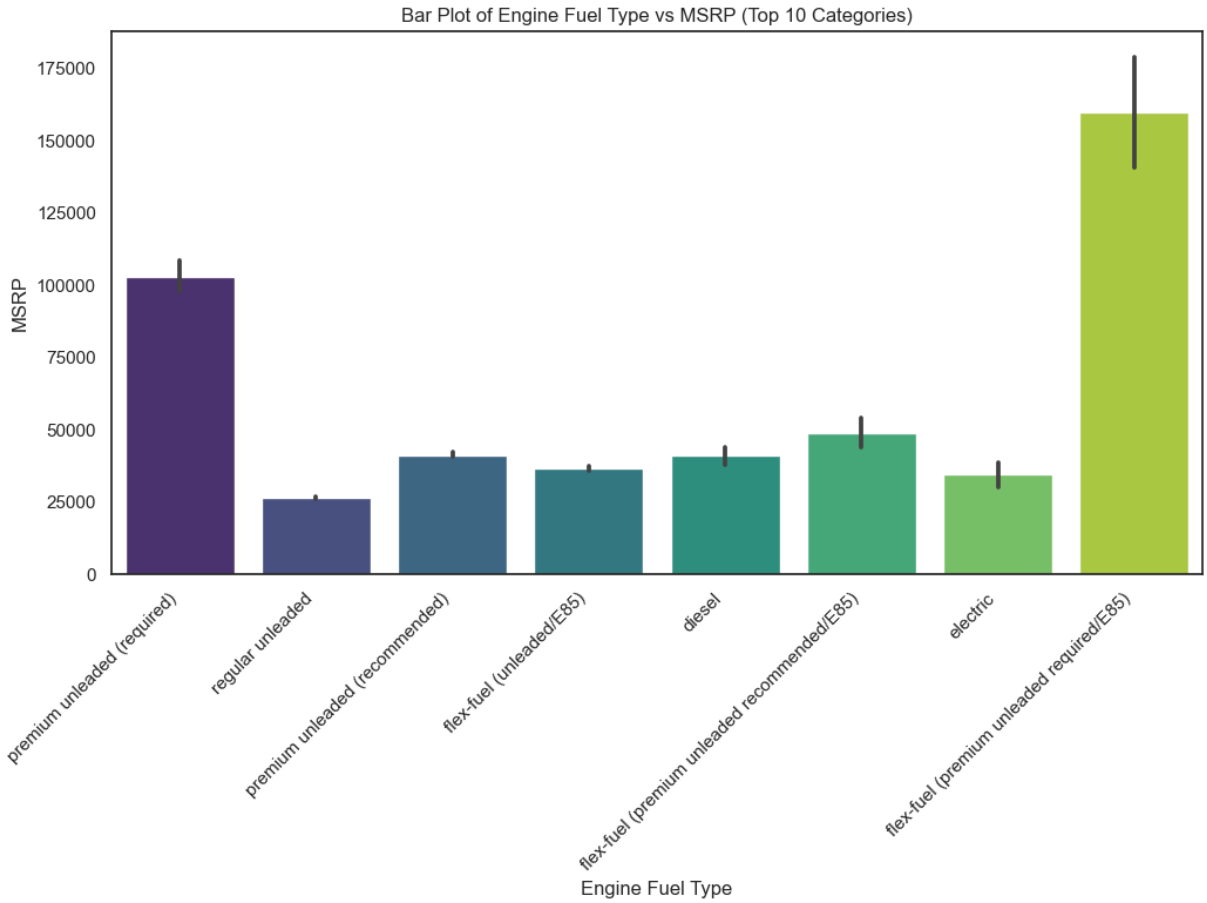
```python
# Bar Plot
plt.figure(figsize=(12, 6))
sns.barplot(x='Model', y='MSRP', data=df_top_model_cat, palette='viridis')
plt.title('Bar Plot of Model vs MSRP (Top 10 Categories)')
plt.xlabel('Model')
plt.ylabel('MSRP')
plt.xticks(rotation=45, ha='right')
plt.show()
#code that i have written ends here

# Select the top 15 categories based on average MSRP for 'Vehicle Style'
top_style_cat = data_car.groupby('Vehicle Style')['MSRP'].mean().sort_values(ascend
df_top_style_cat = data_car[data_car['Vehicle Style'].isin(top_style_cat)]

#code that i have written
# Bar Plot
plt.figure(figsize=(12, 6))
sns.barplot(x='Vehicle Style', y='MSRP', data=df_top_style_cat, palette='viridis')
plt.title('Bar Plot of Vehicle Style vs MSRP (Top 10 Categories)')
plt.xlabel('Vehicle Style')
plt.ylabel('MSRP')
plt.xticks(rotation=45, ha='right')
plt.show()
#code that i have written ends here
```

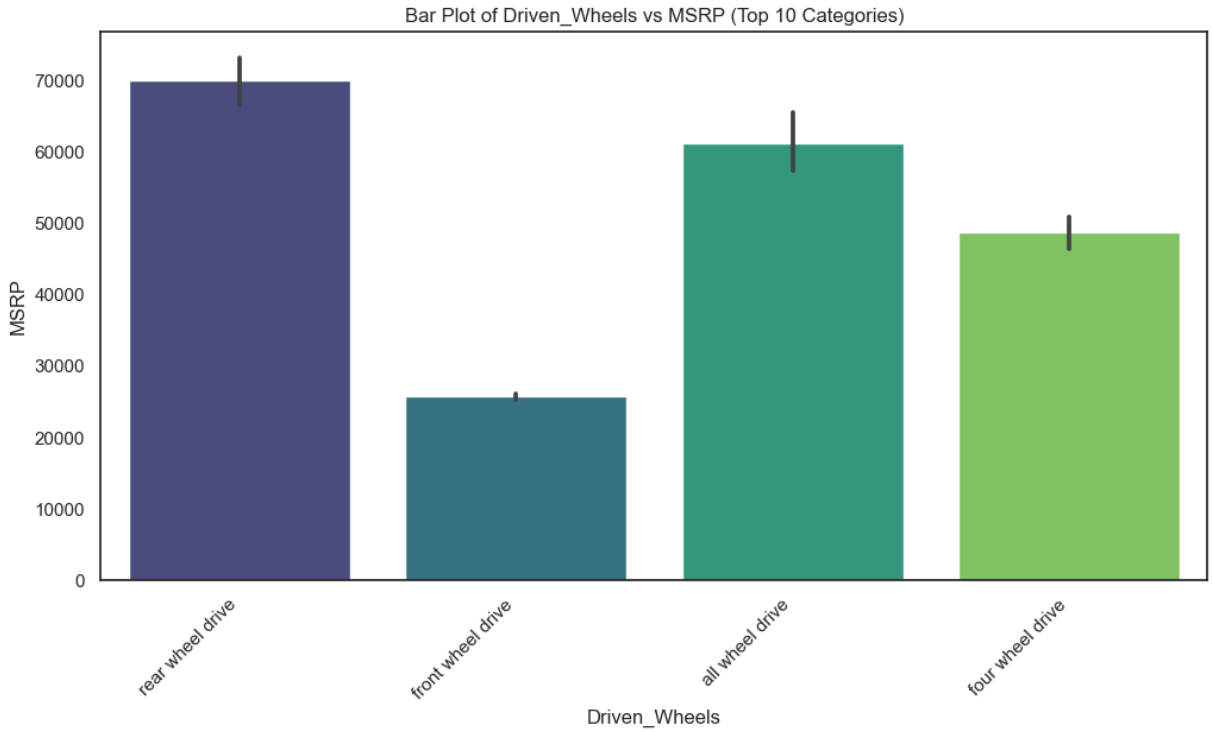


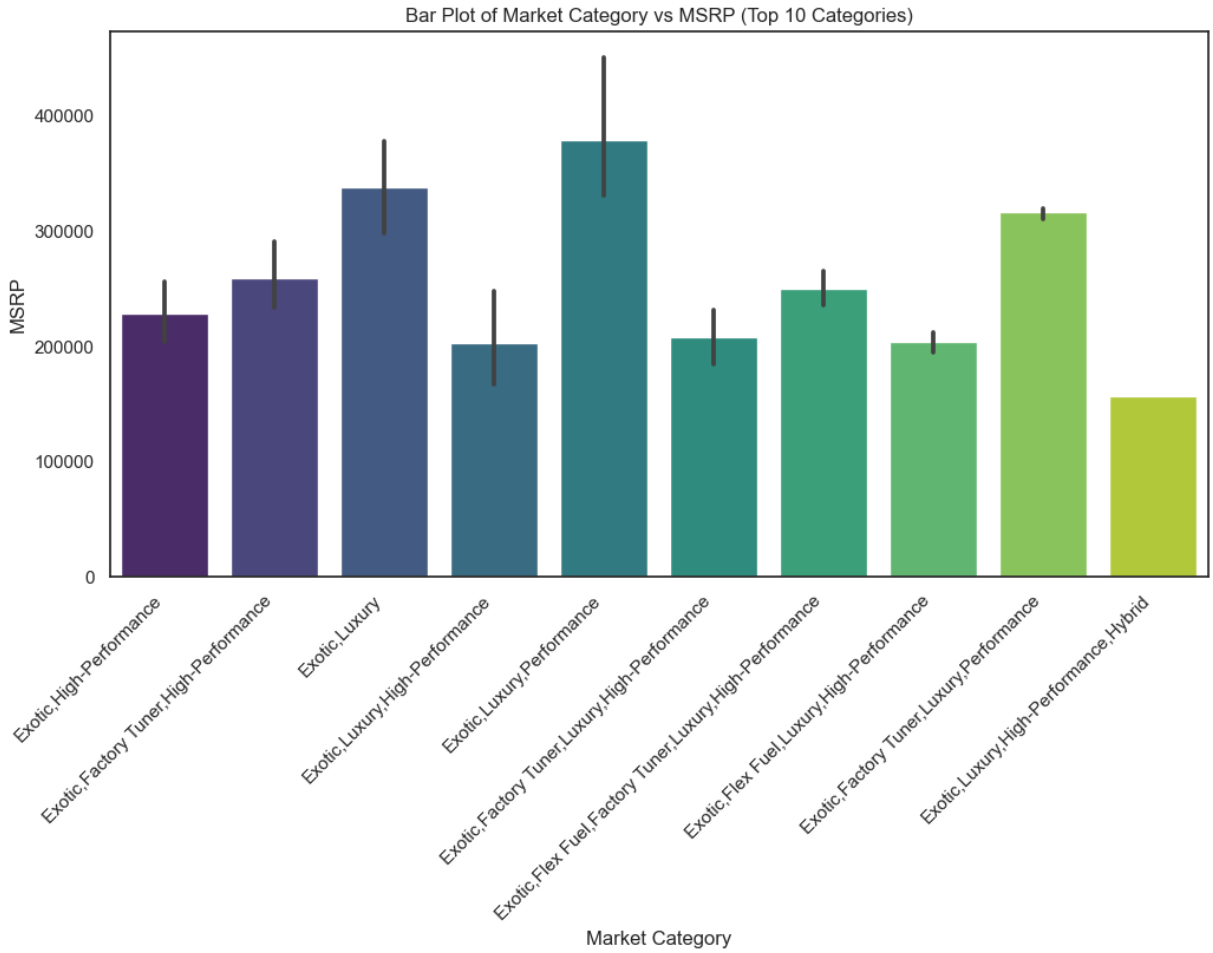


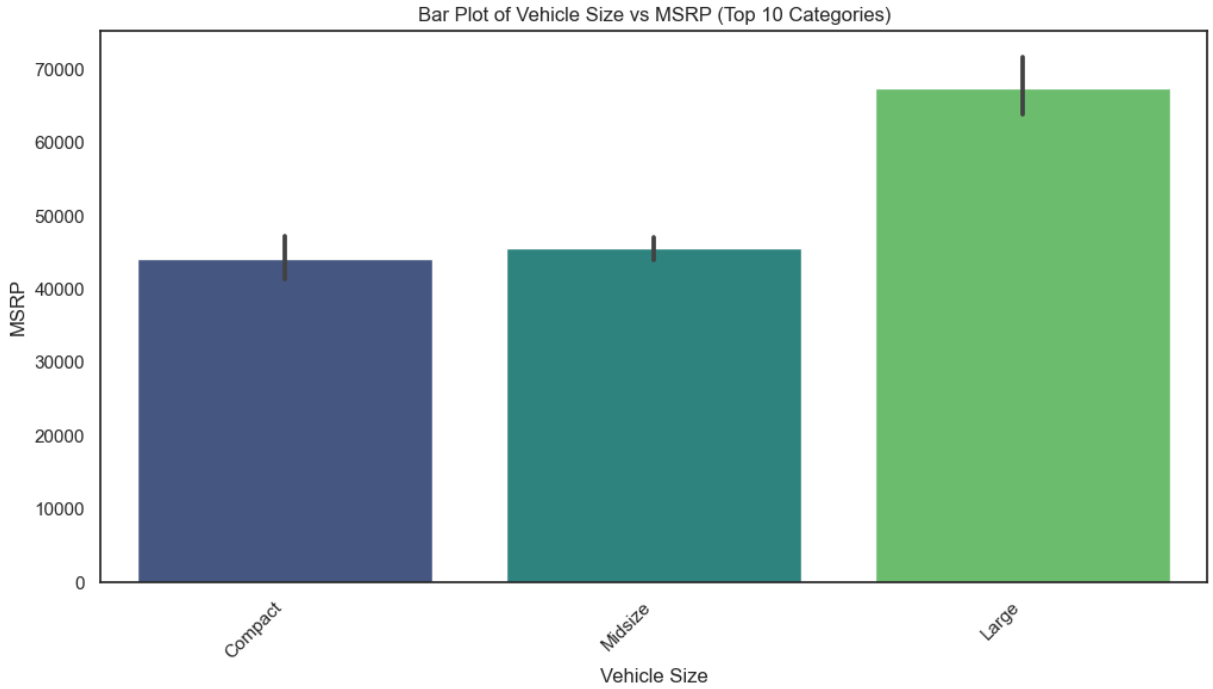
In [317…

```
def create_bar_plot(column_name):
    # Select the top 10 categories based on average MSRP
    top_n_cat = data_car.groupby(column_name)['MSRP'].mean().sort_values(ascending=
    df_top_n_cat = data_car[data_car[column_name].isin(top_n_cat)]

#code that i have written

    # Bar Plot
    plt.figure(figsize=(12, 6))
    sns.barplot(x=column_name, y='MSRP', data=df_top_n_cat, palette='viridis')
    plt.title(f'Bar Plot of {column_name} vs MSRP (Top 10 Categories)')
    plt.xlabel(column_name)
    plt.ylabel('MSRP')
    plt.xticks(rotation=45, ha='right')
    plt.show()

# Create bar plots for each categorical column
create_bar_plot('Engine Fuel Type')
create_bar_plot('Driven_Wheels')
create_bar_plot('Market Category')
create_bar_plot('Vehicle Size')

#code that i have written ends here
```


Bar Plot of Engine Fuel Type vs MSRP (Top 10 Categories)
MSRP
175000
150000
125000
100000
75000
50000
25000
0
premium unleaded (required)
regular unleaded
premium unleaded (recommended)
flex-fuel (unleaded/E85)
diesel
flex-fuel (premium unleaded recommended/E85)
electric
flex-fuel (premium unleaded required/E85)
Engine Fuel Type


Bar Plot of Driven_Wheels vs MSRP (Top 10 Categories)
MSRP
70000
60000
50000
40000
30000
20000
10000
0
rear wheel drive
front wheel drive
all wheel drive
four wheel drive
Driven_Wheels

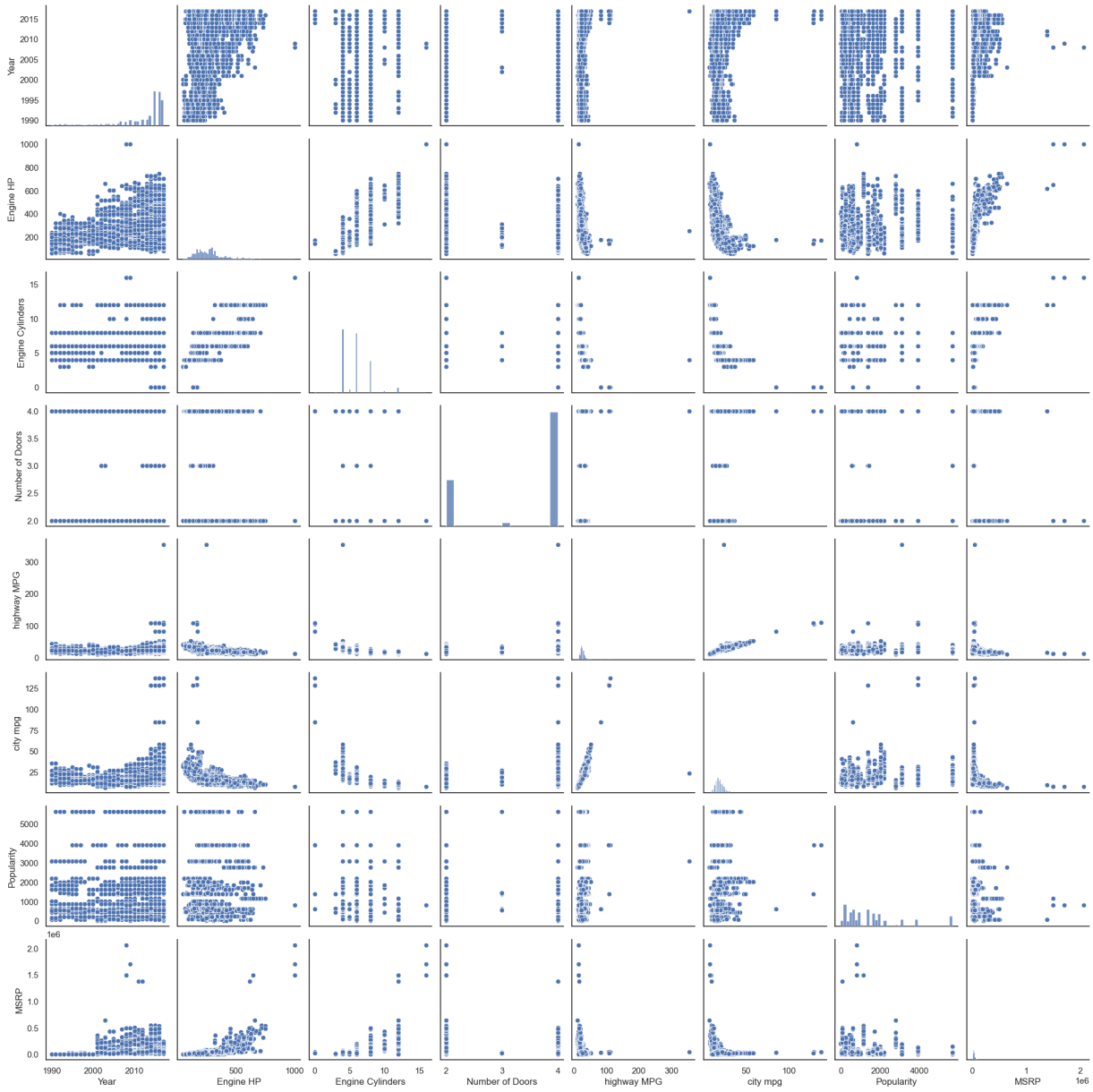




I have also created a pairplot of the columns in the dataframe which shows pairwise relationships among multiple variables. In the context of linear regression, a pair plot is instrumental in assessing the linearity between the target variable and predictor variables, helping to make informed decisions during the modeling process.

In [318…

```
#code that i have written
sns.pairplot(data_car)
plt.show()
```



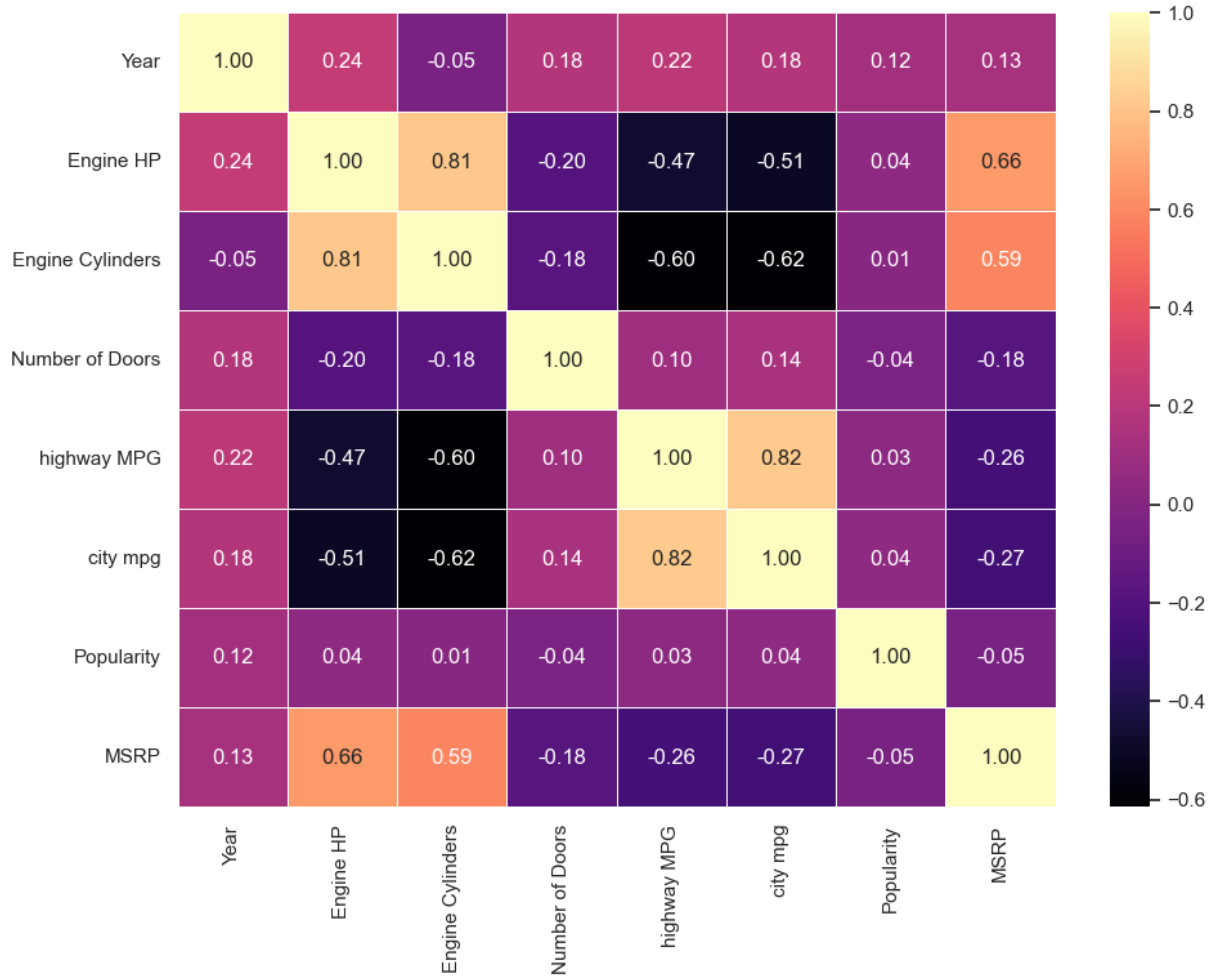
Correlation Matrix

I have created a correlation matrix to better visualize the variables. The significance of a correlation matrix using a heatmap is in its ability to illustrate the direction and intensity of correlations between variables within a dataset. Strong correlations, multicollinearity, pattern recognition, variable selection guidance, and exploratory data analysis are all made easier by it.This helped me to narrow down my variables that affect the target variable and to build a successful model.

In [319…

```
cor_matrix = data_car.corr(numeric_only = True)
plt.figure(figsize=(11, 8))
# Create a heatmap using Seaborn
```

```
sns.heatmap(cor_matrix, annot=True, cmap='magma', fmt='.2f', linewidths=.5)
plt.show()
```



Now we will identify the variabkes that are strongly correlated with the target variable which is price and then will narrow down the variables that are greater then 0.5 as it shows strong positive linear relation with MSRP.

In [320…

```
# Identify features with strong correlation to the MSRP
target_cor = cor_matrix['MSRP'].sort_values(ascending=False)
print("Correlation with 'price':")
print(target_cor)
```

```
Correlation with 'price':
MSRP                1.000000
Engine HP           0.656662
Engine Cylinders    0.589185
Year                0.133684
Popularity         -0.052477
Number of Doors    -0.184699
highway MPG        -0.257585
city mpg           -0.268003
Name: MSRP, dtype: float64
```

In [321…

```
# Select features with correlation above 0.5
sel_features = target_cor[abs(target_cor) > 0.5].index.tolist()
```

```
print("Selected Features:")
print(sel_features)
```

```
Selected Features:
['MSRP', 'Engine HP', 'Engine Cylinders']
```

MSRP will be later dropped as it is the target variable but 'Engine HP' and 'Engine Cylinder' are the variables with a strong positive correlation.

I've carefully gone over the visualizations and chosen a few domain features that I believe have a significant bearing on the Manufacturer's Suggested Retail Price (MSRP). These selections of variables are essential to building our prediction model. Through the integration of the correlation matrix insights and domain-specific features, we have selected a refined set of variables that are critical to effectively estimate automobile pricing. This deliberate choice guarantees that our model is based on the most significant elements, increasing its accuracy and applicability to the automobile sector.

In [322…

```
# Domain knowledge features
domain_feat = ['Year', 'Engine Cylinders', 'highway MPG', 'city mpg']
```

In [323…

```
# Combine selected features from correlation analysis and domain knowledge
final_features = sel_features + domain_features
print("Final Selected Features:")
print(final_features)
#code that i have written ends here
```

```
Final Selected Features:
['MSRP', 'Engine HP', 'Engine Cylinders', 'Year', 'Engine Cylinders', 'highway MPG',
'city mpg']
```

Build your ML (Machine Learning) model

For the training and evaluation of our machine learning model, I employed essential libraries from the Scikit-learn ecosystem.Initial steps involved identifying MSRP as the target variable (y) and segregating it from the features (x).

In [324…

```
# Identify the target variable and features based on the selected features
#code that i have written
target = 'MSRP'
column = data_car[final_features]
```

In [325…

```
# Split the dataset into features and target variable
X = column
y = df[target]
```

The train_test_split facilitated a meticulous division for comprehensive evaluation.

In [326…

```
# Perform the train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

This tell about the dimensions of the training and testing dataset. The displayed shapes, encompassing both the features (X) and the target variable (y), offer transparency into the distribution of data, facilitating a comprehensive evaluation of the model's performance across distinct datasets.

In [327…

```
# Display the shapes of the resulting sets
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

#code that i have written ends here
```

```
X_train shape: (6467, 7)
X_test shape: (1617, 7)
y_train shape: (6467,)
y_test shape: (1617,)
```

Now we will build the model by intializing the model and train the model onto the training data.

In [328…

```
#code that i have written
# Initialize the linear regression model
linear_model = LinearRegression()
```

In [329…

```
# Train the model on the training data
linear_model.fit(X_train, y_train)
```

Out[329]:

```
▾ LinearRegression
LinearRegression()
```

In [330…

```
# Make predictions on the test set
y_predict = linear_model.predict(X_test)
#code that i have written ends here
```

K-fold Cross-Validation is done.

In [342…

```
# Perform k-fold cross-validation (e.g., k=5)
cross_val = cross_val_score(linear_model, X, y, cv=5, scoring='r2')
print("Cross-validated R-squared scores:", cross_val)
print("Mean R-squared:", np.mean(cross_val))
print("Standard Deviation of R-squared:", np.std(cross_val))
```

```
Cross-validated R-squared scores: [1. 1. 1. 1. 1.]
Mean R-squared: 1.0
Standard Deviation of R-squared: 0.0
```

I have also used the three common regression metrics: Mean Squared Error (MSE), Mean Absolute Error (MAE), and R-squared (R2) that assesses the performance of the linear regression model.

In [341…

```
#code that i have written
# Evaluate the model performance
mse = mean_squared_error(y_test, y_predict)
mae = mean_absolute_error(y_test, y_predict)
r2 = r2_score(y_test, y_predict)
print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("R-squared (R2):", r2)
#code that i have written ends here
```

```
Mean Squared Error (MSE): 2.3776819453950907e-22
Mean Absolute Error (MAE): 8.764221671630036e-12
R-squared (R2): 1.0
```

In [338…

```
# Display the coefficients
co_efficients = pd.DataFrame({'Feature': X.columns, 'Coefficient': linear_model.coe
print(co_efficients)
#code that i have written ends here
```

```
           Feature   Coefficient
0             MSRP  1.000000e+00
1         Engine HP -2.778484e-15
2  Engine Cylinders -9.172480e-14
3             Year  5.033899e-14
4  Engine Cylinders -9.172480e-14
5       highway MPG -6.978876e-15
6          city mpg -1.535923e-14
```

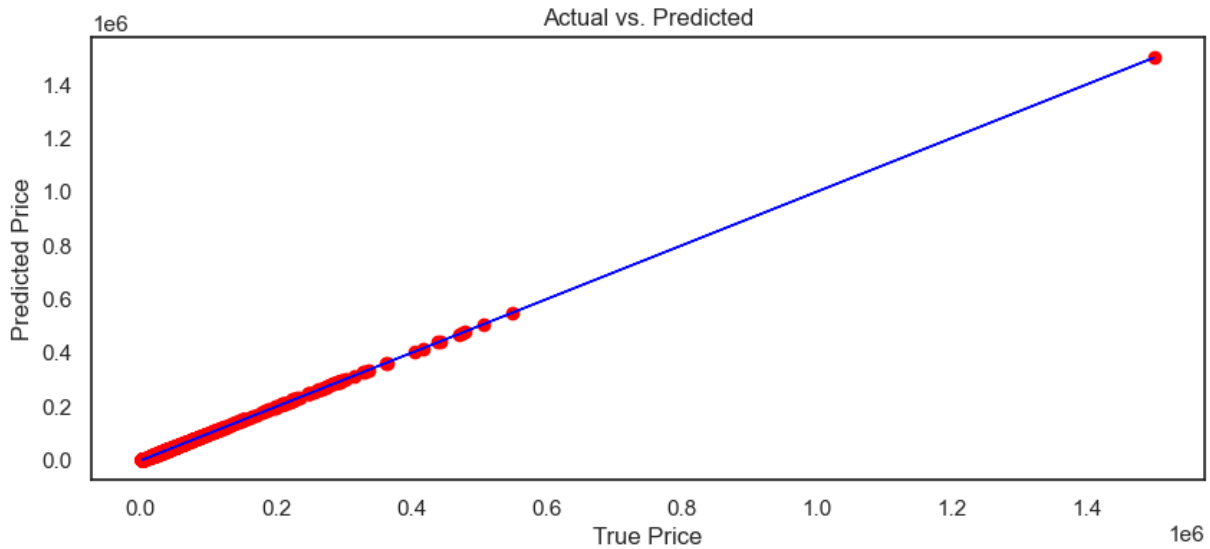
With almost zero MSE and MAE and a flawless R-squared score of 1.0, these excellent assessment metrics suggest that the linear regression model performed exceptionally well on the training set. Minimal mistakes in predicting the MSRP are indicated by low values of MSE and MAE, which demonstrate a high degree of precision in capturing the link between the features and the target variable (MSRP). An ideal fit is shown by the model's R-squared value of 1.0, which shows that it fully explains the variance in the MSRP.

It's important to proceed cautiously when interpreting these data, though. A perfect match on the training set may indicate overfitting, in which case the model has learned the entire training set—noises and outliers included. It is advised to explore new evaluation metrics, make potential improvements to the feature selection process, and validate the model with untested data in order to guarantee its generalizability.

Visualizing the result:

In [337…

```
#code that i have written
plt.figure(figsize=(10,4))
plt.scatter(y_test, y_predict, color = 'red')
plt.xlabel('True Price')
plt.ylabel('Predicted Price')
plt.title('Actual vs. Predicted')
plt.plot(y_test, np.polyval(np.polyfit(y_test, y_predict, 1), y_test), color='blue'
plt.show()
#code that i have written ends here
```



In [336…

```
print("Model training and evaluation completed.")
```

```
Model training and evaluation completed.
```

Feature Engineering

It is used to manipulate the tables allowing us to create some new features as per the requirement. Feature Engineering is required when there are null values in the dataset or are missing values that require us to perform feature engineering to our model but because the r square value is good so and there were no null values as such I didn't apply feature engineering here.

Conclusion

1. Data Preprocessing: I have first preprocessed the data by removing any duplicated and missing values bringing the datset into 1nf to further visualize it.

2. Statistical analysis and Bivariate Analysis: I performed statistical analysis by using the describe function that gave me the statistical evaluation of the columns and i also find percentiles that helped me to understand it better. I performed bivariate analysis by plotting the graph by first converting it into categorical and numerical columns and then plotting both the columns against price to narrow the domain features used later for the model.

3. Linear Regression Using Cross-Fold To strengthen the linear regression model, sophisticated methods such as cross-validation and separating training and testing data were used. The ability to see projected and actual outputs allowed for a concrete evaluation of accuracy. Strict evaluation metrics including MSE, MAE, and R2 highlighted the model's efficacy. When taken as a whole, these methods raised trust in the model's ability to consistently produce precise predictions on data that has never been seen before.

4. Feature Engineering:

5.Final Remarks: After completing this machine-learning project with the car dataset, I was able to identify the critical elements affecting market prices. The linear regression model revealed important insights for stakeholders and was strengthened by thorough assessment and visualization methods. This investigative trip not only deepened our comprehension of price dynamics but also demonstrated how machine learning can effectively decipher complex patterns in the automobile sector.

Refernces:

1. Kaggle link: https://www.kaggle.com/datasets/aminesnoussi/carprice-assignmentcsv/data

In [ ]: